



Laboratorio 02: Redes neuronales poco profundas

Modelos en PyTorch para deterioro cognitivo

Dr. Juan Bekios Calfa

Deep Learning

Universidad Católica del Norte

Publicación: 14/04/2026

Entrega: 13/05/2026



Propósito del laboratorio

Objetivo general

Desarrollar redes neuronales poco profundas en PyTorch para estudiar el deterioro cognitivo mediante seis experimentos independientes, incorporando codificación multilabel tipo one-hot, la metodología CRISP-DM, validación experimental honesta y una medida práctica de incertidumbre.

Propósito del laboratorio

Objetivo general

Desarrollar redes neuronales poco profundas en PyTorch para estudiar el deterioro cognitivo mediante seis experimentos independientes, incorporando codificación multilabel tipo one-hot, la metodología CRISP-DM, validación experimental honesta y una medida práctica de incertidumbre.

Objetivos específicos

- Comprender el problema de deterioro cognitivo como una colección de seis experimentos supervisados.
- Analizar las variables objetivo GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5 como experimentos independientes.

Propósito del laboratorio

Objetivo general

Desarrollar redes neuronales poco profundas en PyTorch para estudiar el deterioro cognitivo mediante seis experimentos independientes, incorporando codificación multilabel tipo one-hot, la metodología CRISP-DM, validación experimental honesta y una medida práctica de incertidumbre.

Objetivos específicos

- Comprender el problema de deterioro cognitivo como una colección de seis experimentos supervisados.
- Analizar las variables objetivo GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5 como experimentos independientes.
- Codificar cada columna objetivo como un vector multilabel tipo one-hot.

Propósito del laboratorio

Objetivo general

Desarrollar redes neuronales poco profundas en PyTorch para estudiar el deterioro cognitivo mediante seis experimentos independientes, incorporando codificación multilabel tipo one-hot, la metodología CRISP-DM, validación experimental honesta y una medida práctica de incertidumbre.

Objetivos específicos

- Comprender el problema de deterioro cognitivo como una colección de seis experimentos supervisados.
- Analizar las variables objetivo GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5 como experimentos independientes.
- Codificar cada columna objetivo como un vector multilabel tipo one-hot.
- Implementar una red neuronal poco profunda en PyTorch cuyo número de salidas dependa del experimento activo.

Propósito del laboratorio

Objetivo general

Desarrollar redes neuronales poco profundas en PyTorch para estudiar el deterioro cognitivo mediante seis experimentos independientes, incorporando codificación multilabel tipo one-hot, la metodología CRISP-DM, validación experimental honesta y una medida práctica de incertidumbre.

Objetivos específicos

- Comprender el problema de deterioro cognitivo como una colección de seis experimentos supervisados.
- Analizar las variables objetivo GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5 como experimentos independientes.
- Codificar cada columna objetivo como un vector multilabel tipo one-hot.
- Implementar una red neuronal poco profunda en PyTorch cuyo número de salidas dependa del experimento activo.
- Incorporar validación interna y externa para reducir sesgo en la selección del modelo.

Propósito del laboratorio

Objetivo general

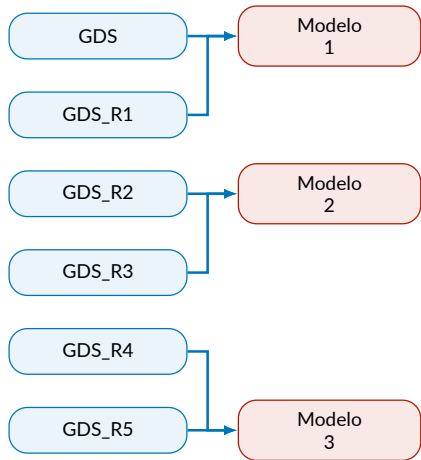
Desarrollar redes neuronales poco profundas en PyTorch para estudiar el deterioro cognitivo mediante seis experimentos independientes, incorporando codificación multilabel tipo one-hot, la metodología CRISP-DM, validación experimental honesta y una medida práctica de incertidumbre.

Objetivos específicos

- Comprender el problema de deterioro cognitivo como una colección de seis experimentos supervisados.
- Analizar las variables objetivo GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5 como experimentos independientes.
- Codificar cada columna objetivo como un vector multilabel tipo one-hot.
- Implementar una red neuronal poco profunda en PyTorch cuyo número de salidas dependa del experimento activo.
- Incorporar validación interna y externa para reducir sesgo en la selección del modelo.
- Estimar incertidumbre predictiva con Monte Carlo Dropout.

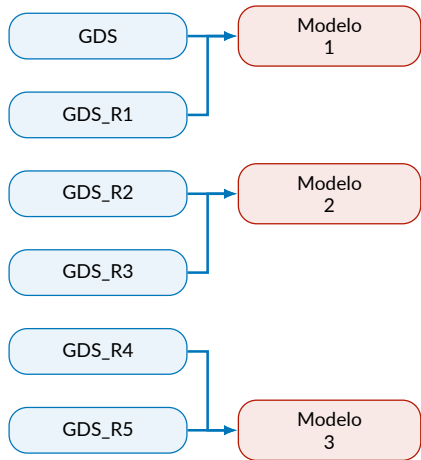
Idea central del laboratorio

- El dataset contiene varias formas de representar el deterioro cognitivo.



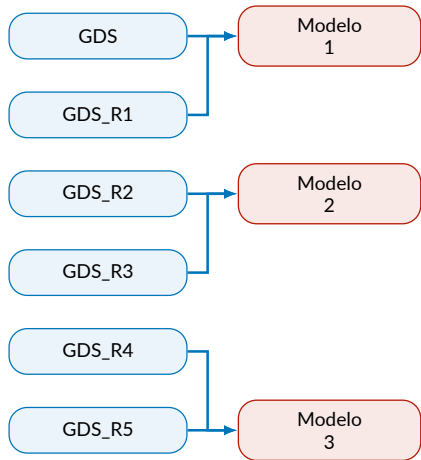
Idea central del laboratorio

- El dataset contiene varias formas de representar el deterioro cognitivo.
- Cada columna objetivo define un experimento distinto, porque cambia el número de clases y su distribución.



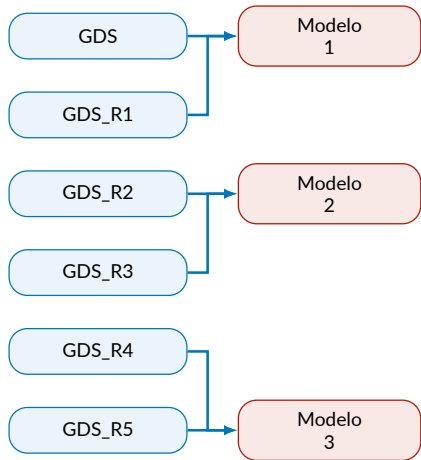
Idea central del laboratorio

- El dataset contiene varias formas de representar el deterioro cognitivo.
- Cada columna objetivo define un experimento distinto, porque cambia el número de clases y su distribución.
- En lugar de entrenar un único modelo con seis salidas, se entrenará un modelo por cada columna objetivo.



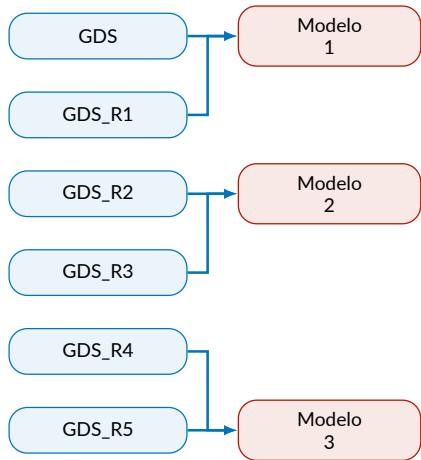
Idea central del laboratorio

- El dataset contiene varias formas de representar el deterioro cognitivo.
- Cada columna objetivo define un experimento distinto, porque cambia el número de clases y su distribución.
- En lugar de entrenar un único modelo con seis salidas, se entrenará un modelo por cada columna objetivo.
- En cada experimento, la clase escalar se transformará a un vector multilabel tipo one-hot antes de entrenar la red.



Idea central del laboratorio

- El dataset contiene varias formas de representar el deterioro cognitivo.
- Cada columna objetivo define un experimento distinto, porque cambia el número de clases y su distribución.
- En lugar de entrenar un único modelo con seis salidas, se entrenará un modelo por cada columna objetivo.
- En cada experimento, la clase escalar se transformará a un vector multilabel tipo one-hot antes de entrenar la red.





Qué significa que la tarea sea multilabel

- En clasificación multiclase, cada muestra pertenece a una sola clase excluyente.

Qué significa que la tarea sea multilabel

- En clasificación multiclase, cada muestra pertenece a una sola clase excluyente.
- En este laboratorio, cada columna objetivo se tomará por separado y su clase se transformará a un vector multilabel tipo one-hot.

Qué significa que la tarea sea multilabel

- En clasificación multiclase, cada muestra pertenece a una sola clase excluyente.
- En este laboratorio, cada columna objetivo se tomará por separado y su clase se transformará a un vector multilabel tipo one-hot.
- Por tanto, la salida del modelo seguirá siendo un vector binario o probabilístico, pero solo para las clases del experimento activo.

Qué significa que la tarea sea multilabel

- En clasificación multiclase, cada muestra pertenece a una sola clase excluyente.
- En este laboratorio, cada columna objetivo se tomará por separado y su clase se transformará a un vector multilabel tipo one-hot.
- Por tanto, la salida del modelo seguirá siendo un vector binario o probabilístico, pero solo para las clases del experimento activo.
- Esto significa que habrá seis modelos distintos: uno para GDS y uno para cada agrupación recodificada.

Qué significa que la tarea sea multilabel

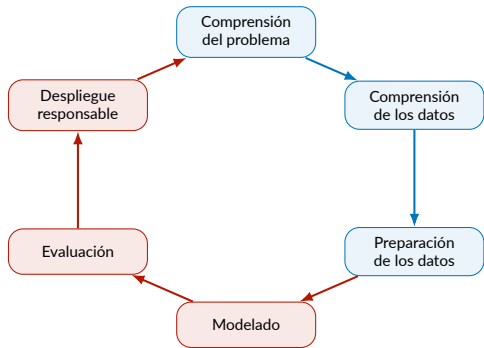
- En clasificación multiclase, cada muestra pertenece a una sola clase excluyente.
- En este laboratorio, cada columna objetivo se tomará por separado y su clase se transformará a un vector multilabel tipo one-hot.
- Por tanto, la salida del modelo seguirá siendo un vector binario o probabilístico, pero solo para las clases del experimento activo.
- Esto significa que habrá seis modelos distintos: uno para GDS y uno para cada agrupación recodificada.

Ejemplo

Si un experimento tiene tres clases y una observación pertenece a la clase 2, entonces su codificación multilabel será $[0, 1, 0]$.

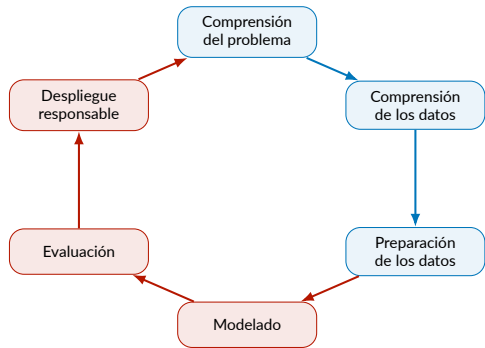
CRISP-DM como estructura del trabajo

1. Comprensión del problema.



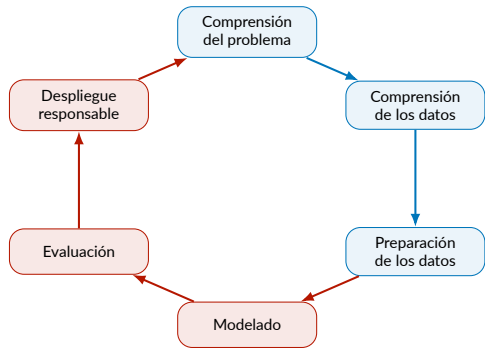
CRISP-DM como estructura del trabajo

1. Comprensión del problema.
2. Comprensión de los datos.



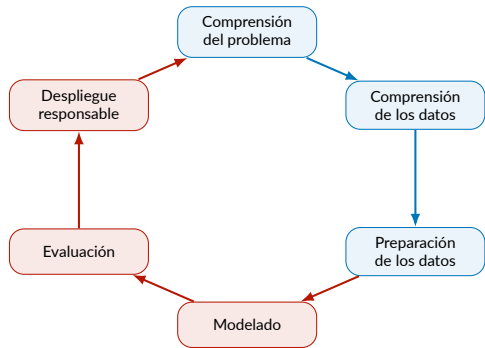
CRISP-DM como estructura del trabajo

1. Comprensión del problema.
2. Comprensión de los datos.
3. Preparación de los datos.



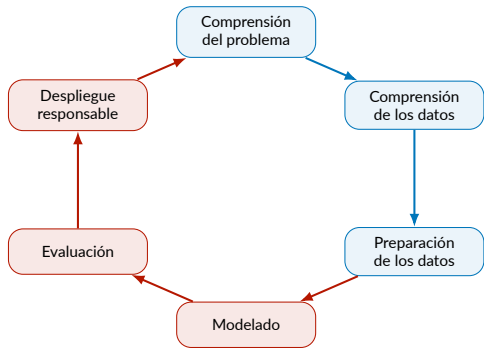
CRISP-DM como estructura del trabajo

1. Comprensión del problema.
2. Comprensión de los datos.
3. Preparación de los datos.
4. Modelado.



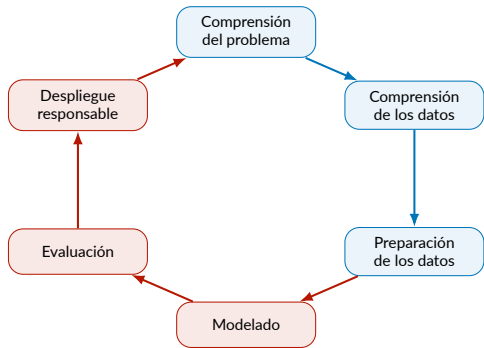
CRISP-DM como estructura del trabajo

1. Comprensión del problema.
2. Comprensión de los datos.
3. Preparación de los datos.
4. Modelado.
5. Evaluación.



CRISP-DM como estructura del trabajo

1. Comprensión del problema.
2. Comprensión de los datos.
3. Preparación de los datos.
4. Modelado.
5. Evaluación.
6. Despliegue o uso responsable.

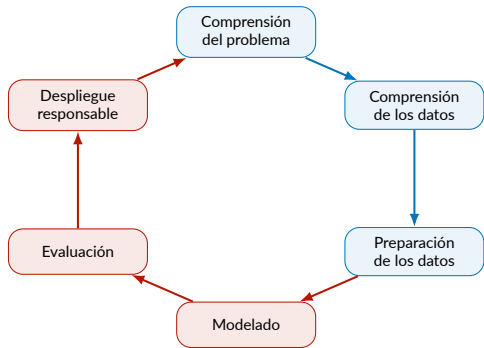


CRISP-DM como estructura del trabajo

1. Comprensión del problema.
2. Comprensión de los datos.
3. Preparación de los datos.
4. Modelado.
5. Evaluación.
6. Despliegue o uso responsable.

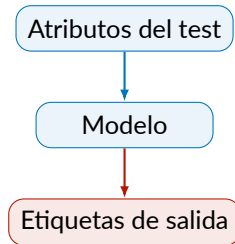
Mensaje principal

La red neuronal es solo una parte del proceso. El diseño del target, la validación y la interpretación son tan importantes como la arquitectura.



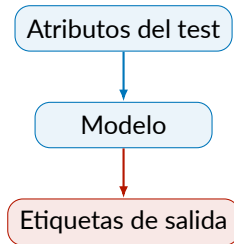
Comprensión del problema

- Se desea predecir niveles de deterioro cognitivo a partir de 15 atributos binarios obtenidos desde un test neuropsicológico.



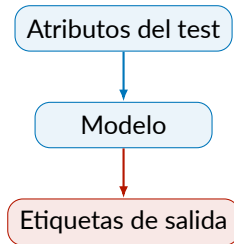
Comprensión del problema

- Se desea predecir niveles de deterioro cognitivo a partir de 15 atributos binarios obtenidos desde un test neuropsicológico.
- El objetivo del laboratorio no es realizar diagnóstico clínico, sino aprender a formular y evaluar modelos computacionales.



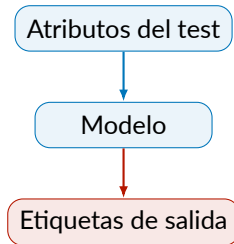
Comprensión del problema

- Se desea predecir niveles de deterioro cognitivo a partir de 15 atributos binarios obtenidos desde un test neuropsicológico.
- El objetivo del laboratorio no es realizar diagnóstico clínico, sino aprender a formular y evaluar modelos computacionales.
- El problema será abordado como una tarea multilabel, donde una muestra puede activar varias etiquetas.



Comprensión del problema

- Se desea predecir niveles de deterioro cognitivo a partir de 15 atributos binarios obtenidos desde un test neuropsicológico.
- El objetivo del laboratorio no es realizar diagnóstico clínico, sino aprender a formular y evaluar modelos computacionales.
- El problema será abordado como una tarea multilabel, donde una muestra puede activar varias etiquetas.
- También es un buen caso para discutir incertidumbre, porque no todas las predicciones deben interpretarse con la misma confianza.





Resumen del dataset utilizado

- El archivo contiene 1119 observaciones.

Resumen del dataset utilizado

- El archivo contiene 1119 observaciones.
- Las variables de entrada corresponden a 15 atributos binarios.

Resumen del dataset utilizado

- El archivo contiene 1119 observaciones.
- Las variables de entrada corresponden a 15 atributos binarios.
- Existe un identificador llamado ID que no debe utilizarse como atributo predictivo.

Resumen del dataset utilizado

- El archivo contiene 1119 observaciones.
- Las variables de entrada corresponden a 15 atributos binarios.
- Existe un identificador llamado ID que no debe utilizarse como atributo predictivo.
- Las variables objetivo disponibles son GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.

Resumen del dataset utilizado

- El archivo contiene 1119 observaciones.
- Las variables de entrada corresponden a 15 atributos binarios.
- Existe un identificador llamado ID que no debe utilizarse como atributo predictivo.
- Las variables objetivo disponibles son GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.
- Cada una de estas columnas se usará en un experimento independiente. La columna GDS se mantendrá como referencia metodológica por su mal condicionamiento.

Atributos de entrada del modelo

Atributos	Tipo
Día, Mes, Año, Estación	Binario
País, Ciudad, CalleLugar, NumeroPiso	Binario
Miguel2, González2, Avenida2, Imperial2	Binario
A682, Caldera2, Copiapo2	Binario

0	1	0	1	0
1	0	1	0	1
0	1	0	1	0

Matriz binaria de entrada

Atributos de entrada del modelo

Atributos	Tipo
Día, Mes, Año, Estación	Binario
País, Ciudad, CalleLugar, NumeroPiso	Binario
Miguel2, González2, Avenida2, Imperial2	Binario
A682, Caldera2, Copiapo2	Binario

0	1	0	1	0
1	0	1	0	1
0	1	0	1	0

Matriz binaria de entrada

Implicancia práctica

Como los atributos son binarios, el preprocesamiento es más simple que en problemas con variables continuas de escalas muy distintas.

Codificación de la salida por experimento

- En cada experimento se toma una sola columna objetivo, por ejemplo GDS_R2.

$$\text{clase} = 2 \implies y = [0, 1, 0]$$

Codificación de la salida por experimento

- En cada experimento se toma una sola columna objetivo, por ejemplo GDS_R2.
- Luego, la clase escalar de esa columna se transforma a un vector multilabel tipo one-hot.

$$\text{clase} = 2 \implies y = [0, 1, 0]$$

Codificación de la salida por experimento

- En cada experimento se toma una sola columna objetivo, por ejemplo GDS_R2.
- Luego, la clase escalar de esa columna se transforma a un vector multilabel tipo one-hot.
- Si el experimento tiene K clases, entonces la salida tendrá K componentes binarias.

$$\text{clase} = 2 \implies y = [0, 1, 0]$$

Codificación de la salida por experimento

- En cada experimento se toma una sola columna objetivo, por ejemplo GDS_R2.
- Luego, la clase escalar de esa columna se transforma a un vector multilabel tipo one-hot.
- Si el experimento tiene K clases, entonces la salida tendrá K componentes binarias.
- Esto permite usar una formulación multilabel aun cuando cada experimento parte desde una sola clase por muestra.

$$\text{clase} = 2 \implies y = [0, 1, 0]$$

Preparación del target multilabel

```
feature_cols = [  
    "Día", "Mes", "Año", "Estación", "País", "Ciudad", "CalleLugar",  
    "NumeroPiso", "Miguel2", "González2", "Avenida2", "Imperial2",  
    "A682", "Caldera2", "Copiapo2"  
]  
  
target_name = "GDS_R2" # Cambiar a GDS, GDS_R1, ..., GDS_R5  
  
X = df[feature_cols].astype("float32").values  
y_raw = df[target_name].astype(int)  
  
classes = sorted(y_raw.unique())  
class_to_idx = {c: i for i, c in enumerate(classes)}  
y_idx = y_raw.map(class_to_idx).values  
  
Y = np.zeros((len(y_idx), len(classes)), dtype=np.float32)  
Y[np.arange(len(y_idx)), y_idx] = 1.0
```

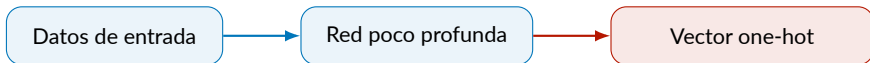
Preparación del target multilabel

```
feature_cols = [  
    "Día", "Mes", "Año", "Estación", "País", "Ciudad", "CalleLugar",  
    "NumeroPiso", "Miguel2", "González2", "Avenida2", "Imperial2",  
    "A682", "Caldera2", "Copiapo2"  
]  
  
target_name = "GDS_R2" # Cambiar a GDS, GDS_R1, ..., GDS_R5  
  
X = df[feature_cols].astype("float32").values  
y_raw = df[target_name].astype(int)  
  
classes = sorted(y_raw.unique())  
class_to_idx = {c: i for i, c in enumerate(classes)}  
y_idx = y_raw.map(class_to_idx).values  
  
Y = np.zeros((len(y_idx), len(classes)), dtype=np.float32)  
Y[np.arange(len(y_idx)), y_idx] = 1.0
```

Resultado esperado

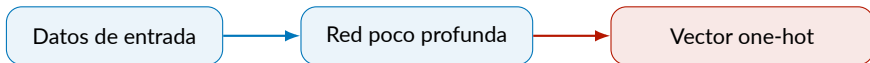
La matriz X contendrá las entradas y la matriz Y contendrá la codificación one-hot multilabel del experimento activo.

Arquitectura general del experimento



- La red tendrá una sola entrada.

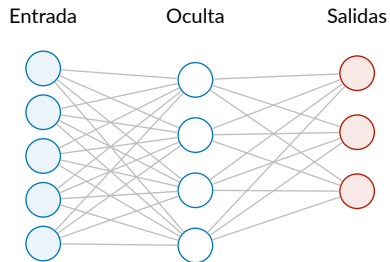
Arquitectura general del experimento



- La red tendrá una sola entrada.
- El número de neuronas de salida dependerá del número de clases del experimento activo.

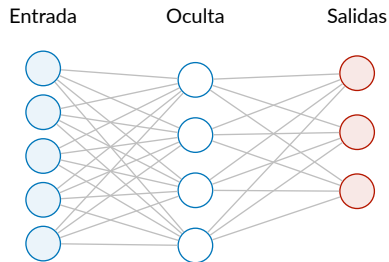
Arquitectura neuronal para clasificación multilabel

- La red tendrá una única entrada con los 15 atributos binarios.



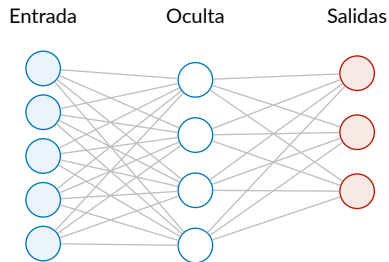
Arquitectura neuronal para clasificación multilabel

- La red tendrá una única entrada con los 15 atributos binarios.
- Tendrá una capa oculta para aprender una representación compartida.



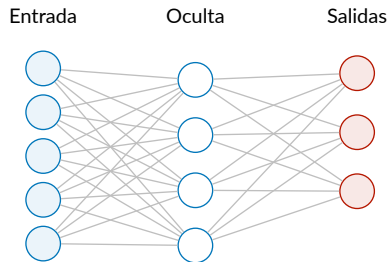
Arquitectura neuronal para clasificación multilabel

- La red tendrá una única entrada con los 15 atributos binarios.
- Tendrá una capa oculta para aprender una representación compartida.
- Tendrá tantas neuronas de salida como clases tenga el experimento activo.



Arquitectura neuronal para clasificación multilabel

- La red tendrá una única entrada con los 15 atributos binarios.
- Tendrá una capa oculta para aprender una representación compartida.
- Tendrá tantas neuronas de salida como clases tenga el experimento activo.
- Cada salida estimará la probabilidad de una componente del vector one-hot.

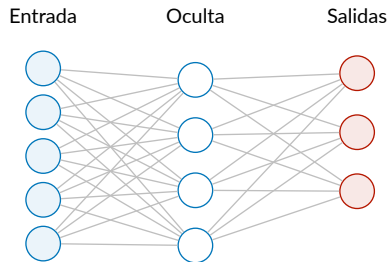


Arquitectura neuronal para clasificación multilabel

- La red tendrá una única entrada con los 15 atributos binarios.
- Tendrá una capa oculta para aprender una representación compartida.
- Tendrá tantas neuronas de salida como clases tenga el experimento activo.
- Cada salida estimará la probabilidad de una componente del vector one-hot.

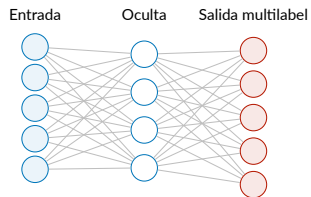
Diferencia con multiclase

Se parte de una sola clase por muestra, pero esa clase se recodifica a formato one-hot multilabel antes del entrenamiento.



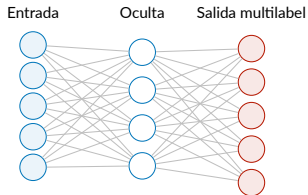
Red neuronal poco profunda

- Una red poco profunda tiene pocas capas ocultas, generalmente una.



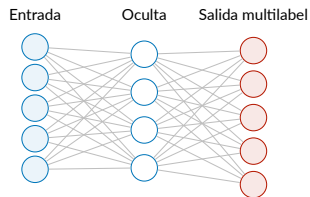
Red neuronal poco profunda

- Una red poco profunda tiene pocas capas ocultas, generalmente una.
- En este laboratorio se busca una arquitectura simple, clara y reproducible.



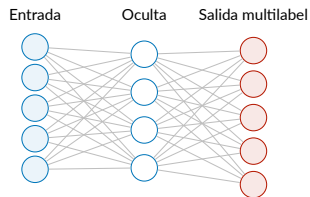
Red neuronal poco profunda

- Una red poco profunda tiene pocas capas ocultas, generalmente una.
- En este laboratorio se busca una arquitectura simple, clara y reproducible.
- El objetivo no es competir con arquitecturas profundas, sino entender el flujo completo de modelado en PyTorch.



Red neuronal poco profunda

- Una red poco profunda tiene pocas capas ocultas, generalmente una.
- En este laboratorio se busca una arquitectura simple, clara y reproducible.
- El objetivo no es competir con arquitecturas profundas, sino entender el flujo completo de modelado en PyTorch.
- También se usará Dropout para mejorar regularización y, más adelante, para estimar incertidumbre.



Definición de la red multilabel en PyTorch

```
import torch
import torch.nn as nn

class ShallowMultiLabelNet(nn.Module):
    def __init__(self, input_dim=15, hidden_dim=32, dropout=0.3, output_dim=3):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        logits = self.fc2(x)
        return logits
```

Interpretación de la arquitectura

- La capa de entrada recibe los 15 atributos binarios.

Interpretación de la arquitectura

- La capa de entrada recibe los 15 atributos binarios.
- La capa oculta aprende una representación intermedia del paciente.

Interpretación de la arquitectura

- La capa de entrada recibe los 15 atributos binarios.
- La capa oculta aprende una representación intermedia del paciente.
- La activación ReLU introduce no linealidad.

Interpretación de la arquitectura

- La capa de entrada recibe los 15 atributos binarios.
- La capa oculta aprende una representación intermedia del paciente.
- La activación ReLU introduce no linealidad.
- Dropout desactiva neuronas aleatoriamente durante el entrenamiento.

Interpretación de la arquitectura

- La capa de entrada recibe los 15 atributos binarios.
- La capa oculta aprende una representación intermedia del paciente.
- La activación ReLU introduce no linealidad.
- Dropout desactiva neuronas aleatoriamente durante el entrenamiento.
- La capa de salida produce un logit por cada componente del vector one-hot del experimento activo.

Cómo se interpreta la salida de la red multilabel

- La red devuelve un logit por cada componente del vector one-hot del experimento activo.

$$\hat{y}_j = \sigma(z_j)$$

Cómo se interpreta la salida de la red multilabel

- La red devuelve un logit por cada componente del vector one-hot del experimento activo.
- A cada logit se le aplica una función sigmoide.

$$\hat{y}_j = \sigma(z_j)$$

Cómo se interpreta la salida de la red multilabel

- La red devuelve un logit por cada componente del vector one-hot del experimento activo.
- A cada logit se le aplica una función sigmoide.
- La sigmoide transforma cada salida en una probabilidad entre 0 y 1.

$$\hat{y}_j = \sigma(z_j)$$

Cómo se interpreta la salida de la red multilabel

- La red devuelve un logit por cada componente del vector one-hot del experimento activo.
- A cada logit se le aplica una función sigmoide.
- La sigmoide transforma cada salida en una probabilidad entre 0 y 1.
- Luego se define un umbral, por ejemplo 0.5, para decidir qué componentes quedan activas.

$$\hat{y}_j = \sigma(z_j)$$

Cómo se interpreta la salida de la red multilabel

- La red devuelve un logit por cada componente del vector one-hot del experimento activo.
- A cada logit se le aplica una función sigmoide.
- La sigmoide transforma cada salida en una probabilidad entre 0 y 1.
- Luego se define un umbral, por ejemplo 0.5, para decidir qué componentes quedan activas.

$$\hat{y}_j = \sigma(z_j)$$

Interpretación

Cada salida se interpreta como la probabilidad de una clase codificada en formato one-hot dentro del experimento activo.



Dataset y DataLoader en PyTorch

```
from torch.utils.data import Dataset

class CognitiveMultiLabelDataset(Dataset):
    def __init__(self, X, Y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.Y = torch.tensor(Y, dtype=torch.float32)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.Y[idx]
```

Dataset y DataLoader en PyTorch

```
from torch.utils.data import Dataset

class CognitiveMultiLabelDataset(Dataset):
    def __init__(self, X, Y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.Y = torch.tensor(Y, dtype=torch.float32)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.Y[idx]
```

Idea práctica

En multilabel, las etiquetas se representan como vectores binarios de tipo `float32`.



Función de pérdida adecuada para multilabel

- En clasificación multiclase se suele usar `CrossEntropyLoss`.

Función de pérdida adecuada para multilabel

- En clasificación multiclase se suele usar `CrossEntropyLoss`.
- En clasificación multilabel se debe usar una pérdida por etiqueta.



Función de pérdida adecuada para multilabel

- En clasificación multiclase se suele usar `CrossEntropyLoss`.
- En clasificación multilabel se debe usar una pérdida por etiqueta.
- Una opción estándar es `BCEWithLogitsLoss`.

Función de pérdida adecuada para multilabel

- En clasificación multiclase se suele usar `CrossEntropyLoss`.
- En clasificación multilabel se debe usar una pérdida por etiqueta.
- Una opción estándar es `BCEWithLogitsLoss`.
- Esta función combina la sigmoide con la entropía cruzada binaria para cada salida.

Función de pérdida adecuada para multilabel

- En clasificación multiclase se suele usar `CrossEntropyLoss`.
- En clasificación multilabel se debe usar una pérdida por etiqueta.
- Una opción estándar es `BCEWithLogitsLoss`.
- Esta función combina la sigmoide con la entropía cruzada binaria para cada salida.

Punto importante

En multilabel, cada neurona de salida aprende una tarea binaria asociada a una etiqueta específica.

Pérdida multilabel con BCEWithLogitsLoss

```
| criterion = nn.BCEWithLogitsLoss()
```

- La salida del modelo debe ser un conjunto de logits.

Pérdida multilabel con BCEWithLogitsLoss

```
criterion = nn.BCEWithLogitsLoss()
```

- La salida del modelo debe ser un conjunto de logits.
- Las etiquetas deben estar codificadas como 0 y 1.

Pérdida multilabel con BCEWithLogitsLoss

```
criterion = nn.BCEWithLogitsLoss()
```

- La salida del modelo debe ser un conjunto de logits.
- Las etiquetas deben estar codificadas como 0 y 1.
- No se debe aplicar sigmoide dentro del modelo si se usa esta pérdida.

Pérdida multilabel con BCEWithLogitsLoss

```
criterion = nn.BCEWithLogitsLoss()
```

- La salida del modelo debe ser un conjunto de logits.
- Las etiquetas deben estar codificadas como 0 y 1.
- No se debe aplicar sigmoide dentro del modelo si se usa esta pérdida.
- La sigmoide se aplica solo al interpretar las salidas.

Entrenamiento de una época

```
def train_one_epoch(model, loader, optimizer, criterion, device="cpu"):
    model.train()
    total_loss = 0.0

    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * xb.size(0)

    return total_loss / len(loader.dataset)
```

Hiperparámetros a explorar

- Tamaño de la capa oculta: 16, 32, 64.

Hidden Dim

Dropout

Learning Rate

Weight Decay

Hiperparámetros a explorar

- Tamaño de la capa oculta: 16, 32, 64.
- Dropout: 0.2, 0.3, 0.5.

Hidden Dim

Dropout

Learning Rate

Weight Decay

Hiperparámetros a explorar

- Tamaño de la capa oculta: 16, 32, 64.
- Dropout: 0.2, 0.3, 0.5.
- Learning Rate: 10^{-2} , 10^{-3} .

Hidden Dim

Dropout

Learning Rate

Weight Decay

Hiperparámetros a explorar

- Tamaño de la capa oculta: 16, 32, 64.
- Dropout: 0.2, 0.3, 0.5.
- Learning Rate: 10^{-2} , 10^{-3} .
- Weight Decay: 0, 10^{-4} .

Hidden Dim

Dropout

Learning Rate

Weight Decay

Hiperparámetros a explorar

- Tamaño de la capa oculta: 16, 32, 64.
- Dropout: 0.2, 0.3, 0.5.
- Learning Rate: 10^{-2} , 10^{-3} .
- Weight Decay: 0, 10^{-4} .
- Batch Size: 16, 32.

Hidden Dim

Dropout

Learning Rate

Weight Decay

Hiperparámetros a explorar

- Tamaño de la capa oculta: 16, 32, 64.
- Dropout: 0.2, 0.3, 0.5.
- Learning Rate: 10^{-2} , 10^{-3} .
- Weight Decay: 0, 10^{-4} .
- Batch Size: 16, 32.

Punto crítico

Estos hiperparámetros no deben elegirse mirando directamente el conjunto de prueba externo.

Hidden Dim

Dropout

Learning Rate

Weight Decay



Por qué no basta con Leave-One-Out

- Leave-One-Out usa casi todos los datos para entrenar y uno para evaluar en cada iteración.

Por qué no basta con Leave-One-Out

- Leave-One-Out usa casi todos los datos para entrenar y uno para evaluar en cada iteración.
- Puede parecer atractivo en datasets pequeños.

Por qué no basta con Leave-One-Out

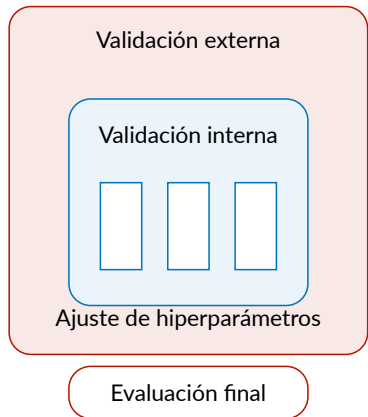
- Leave-One-Out usa casi todos los datos para entrenar y uno para evaluar en cada iteración.
- Puede parecer atractivo en datasets pequeños.
- Sin embargo, si se usa para elegir hiperparámetros y luego reportar desempeño final, puede introducir sesgo.

Por qué no basta con Leave-One-Out

- Leave-One-Out usa casi todos los datos para entrenar y uno para evaluar en cada iteración.
- Puede parecer atractivo en datasets pequeños.
- Sin embargo, si se usa para elegir hiperparámetros y luego reportar desempeño final, puede introducir sesgo.
- El riesgo aumenta cuando se prueban muchas configuraciones o muchas agrupaciones del target.

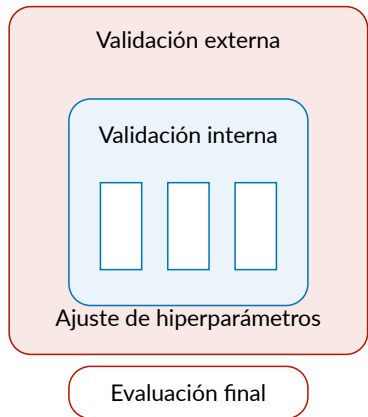
Validación más honesta: esquema anidado

- La validación externa estima el desempeño final del modelo.



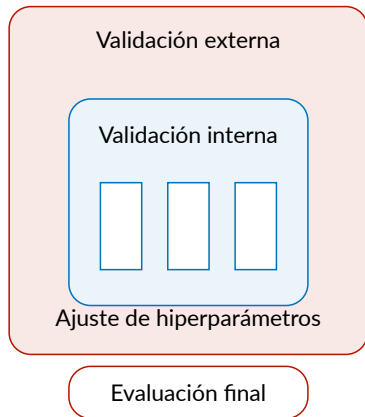
Validación más honesta: esquema anidado

- La validación externa estima el desempeño final del modelo.
- La validación interna se usa para ajustar hiperparámetros.



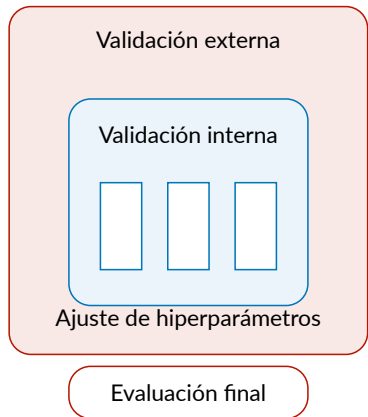
Validación más honesta: esquema anidado

- La validación externa estima el desempeño final del modelo.
- La validación interna se usa para ajustar hiperparámetros.
- Así se separa la selección del modelo de su evaluación final.



Validación más honesta: esquema anidado

- La validación externa estima el desempeño final del modelo.
- La validación interna se usa para ajustar hiperparámetros.
- Así se separa la selección del modelo de su evaluación final.
- Esto reduce el optimismo artificial en los resultados reportados.

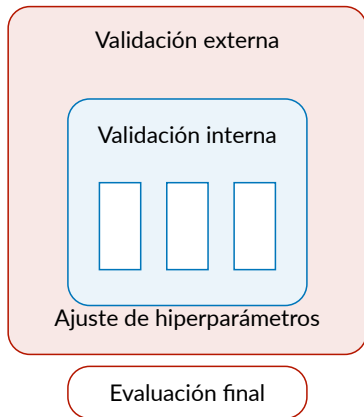


Validación más honesta: esquema anidado

- La validación externa estima el desempeño final del modelo.
- La validación interna se usa para ajustar hiperparámetros.
- Así se separa la selección del modelo de su evaluación final.
- Esto reduce el optimismo artificial en los resultados reportados.

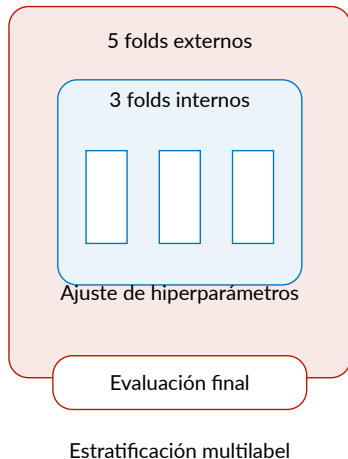
Propuesta del laboratorio

Usar validación externa e interna para ajustar el modelo multilabel y evitar optimismo artificial en la selección de arquitectura e hiperparámetros.



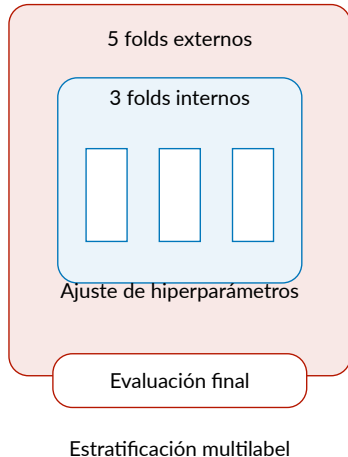
Validación más honesta: esquema propuesto

- Dado que el dataset tiene 1119 muestras y una salida multilabel, se propone usar validación cruzada anidada con partición estratificada multilabel.



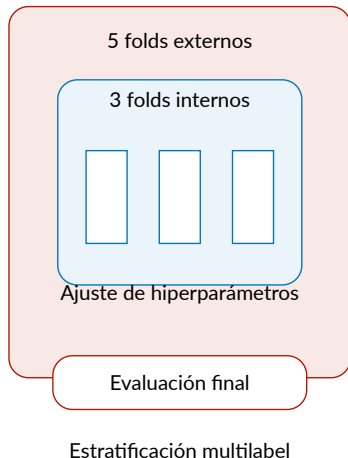
Validación más honesta: esquema propuesto

- Dado que el dataset tiene 1119 muestras y una salida multilabel, se propone usar validación cruzada anidada con partición estratificada multilabel.
- La validación externa puede realizarse con 5 folds para estimar el desempeño final del modelo.



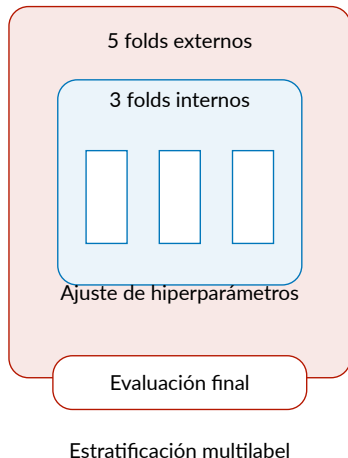
Validación más honesta: esquema propuesto

- Dado que el dataset tiene 1119 muestras y una salida multilabel, se propone usar validación cruzada anidada con partición estratificada multilabel.
- La validación externa puede realizarse con 5 folds para estimar el desempeño final del modelo.
- Dentro de cada fold externo, se propone usar 3 folds internos para seleccionar hiperparámetros.



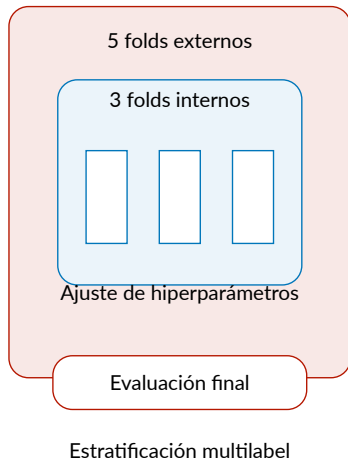
Validación más honesta: esquema propuesto

- Dado que el dataset tiene 1119 muestras y una salida multilabel, se propone usar validación cruzada anidada con partición estratificada multilabel.
- La validación externa puede realizarse con 5 folds para estimar el desempeño final del modelo.
- Dentro de cada fold externo, se propone usar 3 folds internos para seleccionar hiperparámetros.
- Esta estrategia permite separar la selección del modelo de su evaluación final y reduce el optimismo artificial.



Validación más honesta: esquema propuesto

- Dado que el dataset tiene 1119 muestras y una salida multilabel, se propone usar validación cruzada anidada con partición estratificada multilabel.
- La validación externa puede realizarse con 5 folds para estimar el desempeño final del modelo.
- Dentro de cada fold externo, se propone usar 3 folds internos para seleccionar hiperparámetros.
- Esta estrategia permite separar la selección del modelo de su evaluación final y reduce el optimismo artificial.
- La estratificación multilabel busca mantener una distribución similar de etiquetas en todos los folds.

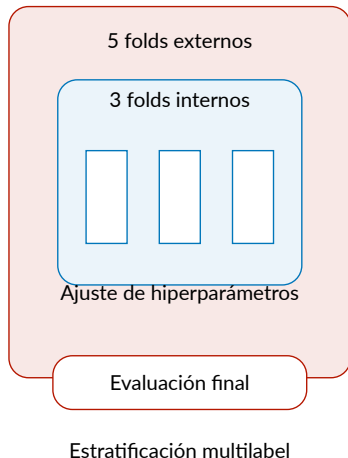


Validación más honesta: esquema propuesto

- Dado que el dataset tiene 1119 muestras y una salida multilabel, se propone usar validación cruzada anidada con partición estratificada multilabel.
- La validación externa puede realizarse con 5 folds para estimar el desempeño final del modelo.
- Dentro de cada fold externo, se propone usar 3 folds internos para seleccionar hiperparámetros.
- Esta estrategia permite separar la selección del modelo de su evaluación final y reduce el optimismo artificial.
- La estratificación multilabel busca mantener una distribución similar de etiquetas en todos los folds.

Propuesta del laboratorio

Usar 5 folds externos y 3 folds internos con estratificación multilabel para ajustar arquitectura, dropout, learning rate, función de activación y umbral de decisión.



Flujo de validación del laboratorio

1. Elegir una columna objetivo, por ejemplo GDS_R2.



Flujo de validación del laboratorio

1. Elegir una columna objetivo, por ejemplo GDS_R2.
2. Transformar esa columna a formato one-hot multilabel.

Flujo de validación del laboratorio

1. Elegir una columna objetivo, por ejemplo GDS_R2.
2. Transformar esa columna a formato one-hot multilabel.
3. Dividir ese experimento en 5 folds externos con estratificación multilabel.

Flujo de validación del laboratorio

1. Elegir una columna objetivo, por ejemplo GDS_R2.
2. Transformar esa columna a formato one-hot multilabel.
3. Dividir ese experimento en 5 folds externos con estratificación multilabel.
4. Dentro del conjunto de entrenamiento externo, aplicar 3 folds internos.

Flujo de validación del laboratorio

1. Elegir una columna objetivo, por ejemplo GDS_R2.
2. Transformar esa columna a formato one-hot multilabel.
3. Dividir ese experimento en 5 folds externos con estratificación multilabel.
4. Dentro del conjunto de entrenamiento externo, aplicar 3 folds internos.
5. Probar distintas configuraciones de hiperparámetros en la validación interna.

Flujo de validación del laboratorio

1. Elegir una columna objetivo, por ejemplo GDS_R2.
2. Transformar esa columna a formato one-hot multilabel.
3. Dividir ese experimento en 5 folds externos con estratificación multilabel.
4. Dentro del conjunto de entrenamiento externo, aplicar 3 folds internos.
5. Probar distintas configuraciones de hiperparámetros en la validación interna.
6. Seleccionar la configuración con mejor desempeño promedio en los folds internos.

Flujo de validación del laboratorio

1. Elegir una columna objetivo, por ejemplo GDS_R2.
2. Transformar esa columna a formato one-hot multilabel.
3. Dividir ese experimento en 5 folds externos con estratificación multilabel.
4. Dentro del conjunto de entrenamiento externo, aplicar 3 folds internos.
5. Probar distintas configuraciones de hiperparámetros en la validación interna.
6. Seleccionar la configuración con mejor desempeño promedio en los folds internos.
7. Reentrenar el modelo con todos los datos del entrenamiento externo.

Flujo de validación del laboratorio

1. Elegir una columna objetivo, por ejemplo GDS_R2.
2. Transformar esa columna a formato one-hot multilabel.
3. Dividir ese experimento en 5 folds externos con estratificación multilabel.
4. Dentro del conjunto de entrenamiento externo, aplicar 3 folds internos.
5. Probar distintas configuraciones de hiperparámetros en la validación interna.
6. Seleccionar la configuración con mejor desempeño promedio en los folds internos.
7. Reentrenar el modelo con todos los datos del entrenamiento externo.
8. Evaluar el modelo final en el fold externo de prueba.

Flujo de validación del laboratorio

1. Elegir una columna objetivo, por ejemplo GDS_R2.
2. Transformar esa columna a formato one-hot multilabel.
3. Dividir ese experimento en 5 folds externos con estratificación multilabel.
4. Dentro del conjunto de entrenamiento externo, aplicar 3 folds internos.
5. Probar distintas configuraciones de hiperparámetros en la validación interna.
6. Seleccionar la configuración con mejor desempeño promedio en los folds internos.
7. Reentrenar el modelo con todos los datos del entrenamiento externo.
8. Evaluar el modelo final en el fold externo de prueba.
9. Repetir el proceso para los seis experimentos, incluyendo GDS como referencia.

Ventaja

Este procedimiento entrega una estimación más honesta del rendimiento del modelo y reduce el riesgo de sobreajuste a la validación.

Métricas recomendadas en multilabel

- En problemas multilabel, cada muestra puede activar varias etiquetas al mismo tiempo.

Métricas recomendadas en multilabel

- En problemas multilabel, cada muestra puede activar varias etiquetas al mismo tiempo.
- Por ello, no basta con usar una sola métrica tradicional de clasificación.

Métricas recomendadas en multilabel

- En problemas multilabel, cada muestra puede activar varias etiquetas al mismo tiempo.
- Por ello, no basta con usar una sola métrica tradicional de clasificación.
- Es necesario evaluar tanto errores por etiqueta como desempeño global del modelo.

Advertencia

En problemas multilabel no basta con Accuracy tradicional, porque una muestra puede activar varias etiquetas a la vez.

Métricas recomendadas en multilabel

- En problemas multilabel, cada muestra puede activar varias etiquetas al mismo tiempo.
- Por ello, no basta con usar una sola métrica tradicional de clasificación.
- Es necesario evaluar tanto errores por etiqueta como desempeño global del modelo.
- En este laboratorio se recomienda analizar varias métricas complementarias.

Advertencia

En problemas multilabel no basta con Accuracy tradicional, porque una muestra puede activar varias etiquetas a la vez.

Métricas recomendadas en multilabel

- En problemas multilabel, cada muestra puede activar varias etiquetas al mismo tiempo.
- Por ello, no basta con usar una sola métrica tradicional de clasificación.
- Es necesario evaluar tanto errores por etiqueta como desempeño global del modelo.
- En este laboratorio se recomienda analizar varias métricas complementarias.

Advertencia

En problemas multilabel no basta con Accuracy tradicional, porque una muestra puede activar varias etiquetas a la vez.

Métricas sugeridas

Hamming Loss, Precision Micro, Recall Micro, F1 Micro, F1 Macro y Exact Match.

Hamming Loss y Exact Match

- Hamming Loss mide la proporción de etiquetas mal clasificadas.

$$\text{Hamming Loss} = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \mathbf{1}(y_{ij} \neq \hat{y}_{ij})$$

$$\text{Exact Match} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(y_i = \hat{y}_i)$$

Hamming Loss y Exact Match

- **Hamming Loss** mide la proporción de etiquetas mal clasificadas.
- Penaliza tanto falsos positivos como falsos negativos en cada etiqueta.

$$\text{Hamming Loss} = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \mathbf{1}(y_{ij} \neq \hat{y}_{ij})$$

$$\text{Exact Match} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\mathbf{y}_i = \hat{\mathbf{y}}_i)$$

Hamming Loss y Exact Match

- **Hamming Loss** mide la proporción de etiquetas mal clasificadas.
- Penaliza tanto falsos positivos como falsos negativos en cada etiqueta.
- Mientras más cercano a 0, mejor es el modelo.

$$\text{Hamming Loss} = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \mathbf{1}(y_{ij} \neq \hat{y}_{ij})$$

$$\text{Exact Match} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\mathbf{y}_i = \hat{\mathbf{y}}_i)$$

Hamming Loss y Exact Match

- **Hamming Loss** mide la proporción de etiquetas mal clasificadas.
- Penaliza tanto falsos positivos como falsos negativos en cada etiqueta.
- Mientras más cercano a 0, mejor es el modelo.

$$\text{Hamming Loss} = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \mathbf{1}(y_{ij} \neq \hat{y}_{ij})$$

- Aquí, N es el número de muestras y L es el número de etiquetas.

$$\text{Exact Match} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\mathbf{y}_i = \hat{\mathbf{y}}_i)$$

Hamming Loss y Exact Match

- **Hamming Loss** mide la proporción de etiquetas mal clasificadas.
- Penaliza tanto falsos positivos como falsos negativos en cada etiqueta.
- Mientras más cercano a 0, mejor es el modelo.

$$\text{Hamming Loss} = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \mathbf{1}(y_{ij} \neq \hat{y}_{ij})$$

- Aquí, N es el número de muestras y L es el número de etiquetas.
- **Exact Match** es una métrica más estricta: exige que todas las etiquetas de una muestra sean correctas simultáneamente.

$$\text{Exact Match} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\mathbf{y}_i = \hat{\mathbf{y}}_i)$$

Hamming Loss y Exact Match

- **Hamming Loss** mide la proporción de etiquetas mal clasificadas.
- Penaliza tanto falsos positivos como falsos negativos en cada etiqueta.
- Mientras más cercano a 0, mejor es el modelo.

$$\text{Hamming Loss} = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \mathbf{1}(y_{ij} \neq \hat{y}_{ij})$$

- Aquí, N es el número de muestras y L es el número de etiquetas.
- **Exact Match** es una métrica más estricta: exige que todas las etiquetas de una muestra sean correctas simultáneamente.

$$\text{Exact Match} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\mathbf{y}_i = \hat{\mathbf{y}}_i)$$

Hamming Loss y Exact Match

- **Hamming Loss** mide la proporción de etiquetas mal clasificadas.
- Penaliza tanto falsos positivos como falsos negativos en cada etiqueta.
- Mientras más cercano a 0, mejor es el modelo.

$$\text{Hamming Loss} = \frac{1}{N \cdot L} \sum_{i=1}^N \sum_{j=1}^L \mathbf{1}(y_{ij} \neq \hat{y}_{ij})$$

- Aquí, N es el número de muestras y L es el número de etiquetas.
- **Exact Match** es una métrica más estricta: exige que todas las etiquetas de una muestra sean correctas simultáneamente.

$$\text{Exact Match} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\mathbf{y}_i = \hat{\mathbf{y}}_i)$$

Interpretación

Hamming Loss evalúa errores etiqueta por etiqueta, mientras que Exact Match exige coincidencia total del vector multilabel.

Precision Micro y Recall Micro

- **Precision Micro** mide qué fracción de las etiquetas predichas como positivas realmente eran positivas.

$$\text{Precision}_{micro} = \frac{\sum TP}{\sum TP + \sum FP}$$

$$\text{Recall}_{micro} = \frac{\sum TP}{\sum TP + \sum FN}$$

Precision Micro y Recall Micro

- **Precision Micro** mide qué fracción de las etiquetas predichas como positivas realmente eran positivas.
- Es útil cuando interesa controlar falsos positivos.

$$\text{Precision}_{micro} = \frac{\sum TP}{\sum TP + \sum FP}$$

$$\text{Recall}_{micro} = \frac{\sum TP}{\sum TP + \sum FN}$$

Precision Micro y Recall Micro

- **Precision Micro** mide qué fracción de las etiquetas predichas como positivas realmente eran positivas.
- Es útil cuando interesa controlar falsos positivos.

$$\text{Precision}_{micro} = \frac{\sum TP}{\sum TP + \sum FP}$$

- **Recall Micro** mide qué fracción de las etiquetas positivas reales fue detectada por el modelo.

$$\text{Recall}_{micro} = \frac{\sum TP}{\sum TP + \sum FN}$$

Precision Micro y Recall Micro

- **Precision Micro** mide qué fracción de las etiquetas predichas como positivas realmente eran positivas.
- Es útil cuando interesa controlar falsos positivos.

$$\text{Precision}_{micro} = \frac{\sum TP}{\sum TP + \sum FP}$$

- **Recall Micro** mide qué fracción de las etiquetas positivas reales fue detectada por el modelo.
- Es útil cuando interesa no perder etiquetas verdaderas.

$$\text{Recall}_{micro} = \frac{\sum TP}{\sum TP + \sum FN}$$

Precision Micro y Recall Micro

- **Precision Micro** mide qué fracción de las etiquetas predichas como positivas realmente eran positivas.
- Es útil cuando interesa controlar falsos positivos.

$$\text{Precision}_{micro} = \frac{\sum TP}{\sum TP + \sum FP}$$

- **Recall Micro** mide qué fracción de las etiquetas positivas reales fue detectada por el modelo.
- Es útil cuando interesa no perder etiquetas verdaderas.

$$\text{Recall}_{micro} = \frac{\sum TP}{\sum TP + \sum FN}$$

- En ambas fórmulas, los verdaderos positivos, falsos positivos y falsos negativos se suman sobre todas las etiquetas y muestras.

Precision Micro y Recall Micro

- **Precision Micro** mide qué fracción de las etiquetas predichas como positivas realmente eran positivas.
- Es útil cuando interesa controlar falsos positivos.

$$\text{Precision}_{micro} = \frac{\sum TP}{\sum TP + \sum FP}$$

- **Recall Micro** mide qué fracción de las etiquetas positivas reales fue detectada por el modelo.
- Es útil cuando interesa no perder etiquetas verdaderas.

$$\text{Recall}_{micro} = \frac{\sum TP}{\sum TP + \sum FN}$$

- En ambas fórmulas, los verdaderos positivos, falsos positivos y falsos negativos se suman sobre todas las etiquetas y muestras.

Precision Micro y Recall Micro

- **Precision Micro** mide qué fracción de las etiquetas predichas como positivas realmente eran positivas.
- Es útil cuando interesa controlar falsos positivos.

$$\text{Precision}_{micro} = \frac{\sum TP}{\sum TP + \sum FP}$$

- **Recall Micro** mide qué fracción de las etiquetas positivas reales fue detectada por el modelo.
- Es útil cuando interesa no perder etiquetas verdaderas.

$$\text{Recall}_{micro} = \frac{\sum TP}{\sum TP + \sum FN}$$

- En ambas fórmulas, los verdaderos positivos, falsos positivos y falsos negativos se suman sobre todas las etiquetas y muestras.

Interpretación

Precision Micro favorece precisión global en etiquetas positivas, mientras Recall Micro favorece cobertura global de etiquetas verdaderas.

F1 Micro y F1 Macro

- **F1 Micro** combina Precision Micro y Recall Micro en una sola medida armónica.

$$F1_{micro} = 2 \cdot \frac{\text{Precision}_{micro} \cdot \text{Recall}_{micro}}{\text{Precision}_{micro} + \text{Recall}_{micro}}$$

$$F1_{macro} = \frac{1}{L} \sum_{j=1}^L F1_j$$

F1 Micro y F1 Macro

- **F1 Micro** combina Precision Micro y Recall Micro en una sola medida armónica.
- Resume el rendimiento global del modelo considerando todas las etiquetas en conjunto.

$$F1_{micro} = 2 \cdot \frac{\text{Precision}_{micro} \cdot \text{Recall}_{micro}}{\text{Precision}_{micro} + \text{Recall}_{micro}}$$

$$F1_{macro} = \frac{1}{L} \sum_{j=1}^L F1_j$$

F1 Micro y F1 Macro

- **F1 Micro** combina Precision Micro y Recall Micro en una sola medida armónica.
- Resume el rendimiento global del modelo considerando todas las etiquetas en conjunto.

$$F1_{micro} = 2 \cdot \frac{\text{Precision}_{micro} \cdot \text{Recall}_{micro}}{\text{Precision}_{micro} + \text{Recall}_{micro}}$$

- **F1 Macro** calcula el F1 por cada etiqueta y luego promedia esos valores.

$$F1_{macro} = \frac{1}{L} \sum_{j=1}^L F1_j$$

F1 Micro y F1 Macro

- **F1 Micro** combina Precision Micro y Recall Micro en una sola medida armónica.
- Resume el rendimiento global del modelo considerando todas las etiquetas en conjunto.

$$F1_{micro} = 2 \cdot \frac{\text{Precision}_{micro} \cdot \text{Recall}_{micro}}{\text{Precision}_{micro} + \text{Recall}_{micro}}$$

- **F1 Macro** calcula el F1 por cada etiqueta y luego promedia esos valores.
- Da el mismo peso a cada etiqueta, incluso si algunas son poco frecuentes.

$$F1_{macro} = \frac{1}{L} \sum_{j=1}^L F1_j$$

F1 Micro y F1 Macro

- **F1 Micro** combina Precision Micro y Recall Micro en una sola medida armónica.
- Resume el rendimiento global del modelo considerando todas las etiquetas en conjunto.

$$F1_{micro} = 2 \cdot \frac{\text{Precision}_{micro} \cdot \text{Recall}_{micro}}{\text{Precision}_{micro} + \text{Recall}_{micro}}$$

- **F1 Macro** calcula el F1 por cada etiqueta y luego promedia esos valores.
- Da el mismo peso a cada etiqueta, incluso si algunas son poco frecuentes.

$$F1_{macro} = \frac{1}{L} \sum_{j=1}^L F1_j$$

- Aquí, L representa el número total de etiquetas.

F1 Micro y F1 Macro

- **F1 Micro** combina Precision Micro y Recall Micro en una sola medida armónica.
- Resume el rendimiento global del modelo considerando todas las etiquetas en conjunto.

$$F1_{micro} = 2 \cdot \frac{\text{Precision}_{micro} \cdot \text{Recall}_{micro}}{\text{Precision}_{micro} + \text{Recall}_{micro}}$$

- **F1 Macro** calcula el F1 por cada etiqueta y luego promedia esos valores.
- Da el mismo peso a cada etiqueta, incluso si algunas son poco frecuentes.

$$F1_{macro} = \frac{1}{L} \sum_{j=1}^L F1_j$$

- Aquí, L representa el número total de etiquetas.

F1 Micro y F1 Macro

- **F1 Micro** combina Precision Micro y Recall Micro en una sola medida armónica.
- Resume el rendimiento global del modelo considerando todas las etiquetas en conjunto.

$$F1_{micro} = 2 \cdot \frac{\text{Precision}_{micro} \cdot \text{Recall}_{micro}}{\text{Precision}_{micro} + \text{Recall}_{micro}}$$

- **F1 Macro** calcula el F1 por cada etiqueta y luego promedia esos valores.
- Da el mismo peso a cada etiqueta, incluso si algunas son poco frecuentes.

$$F1_{macro} = \frac{1}{L} \sum_{j=1}^L F1_j$$

- Aquí, L representa el número total de etiquetas.

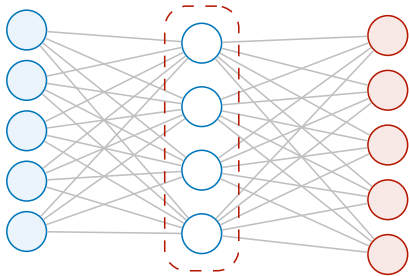
Interpretación

F1 Micro refleja el rendimiento global del sistema, mientras F1 Macro permite detectar si el modelo funciona mal en etiquetas minoritarias.

Arquitectura con incertidumbre

- Para incorporar incertidumbre se mantendrá la capa Dropout en la red.

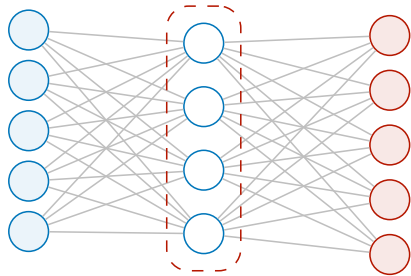
Dropout



Arquitectura con incertidumbre

- Para incorporar incertidumbre se mantendrá la capa Dropout en la red.
- Durante el entrenamiento, Dropout regulariza el modelo.

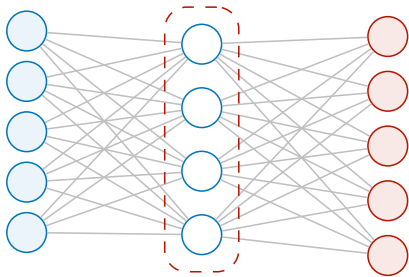
Dropout



Arquitectura con incertidumbre

- Para incorporar incertidumbre se mantendrá la capa Dropout en la red.
- Durante el entrenamiento, Dropout regulariza el modelo.
- Durante la inferencia, si Dropout se mantiene activo, cada pasada genera una predicción levemente distinta.

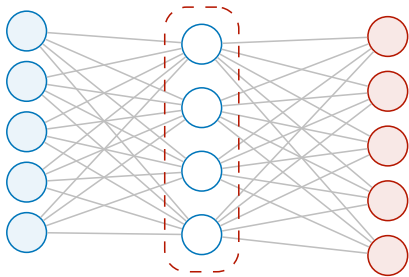
Dropout



Arquitectura con incertidumbre

- Para incorporar incertidumbre se mantendrá la capa Dropout en la red.
- Durante el entrenamiento, Dropout regulariza el modelo.
- Durante la inferencia, si Dropout se mantiene activo, cada pasada genera una predicción levemente distinta.
- Repitiendo varias pasadas, se puede estimar una media y una desviación por etiqueta.

Dropout



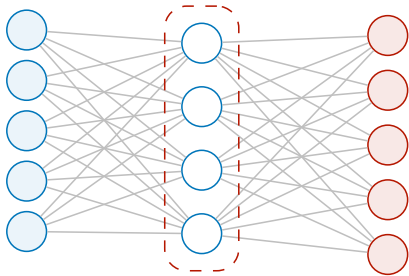
Arquitectura con incertidumbre

- Para incorporar incertidumbre se mantendrá la capa Dropout en la red.
- Durante el entrenamiento, Dropout regulariza el modelo.
- Durante la inferencia, si Dropout se mantiene activo, cada pasada genera una predicción levemente distinta.
- Repitiendo varias pasadas, se puede estimar una media y una desviación por etiqueta.

Interpretación

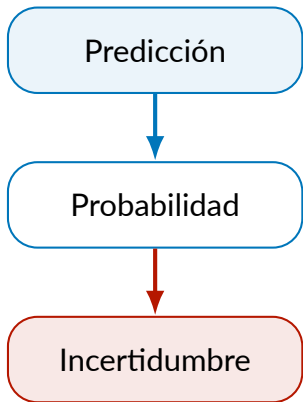
Una gran dispersión entre pasadas sugiere mayor incertidumbre del modelo sobre esa etiqueta.

Dropout



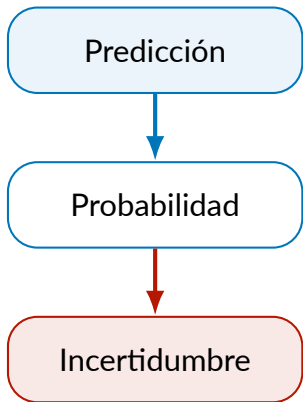
Por qué introducir incertidumbre

- Dos pacientes pueden recibir el mismo vector de etiquetas predichas, pero con confianzas muy distintas.



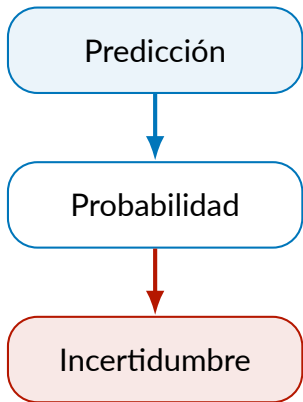
Por qué introducir incertidumbre

- Dos pacientes pueden recibir el mismo vector de etiquetas predichas, pero con confianzas muy distintas.
- En problemas sensibles, una predicción debe acompañarse de una medida de estabilidad o duda del modelo.



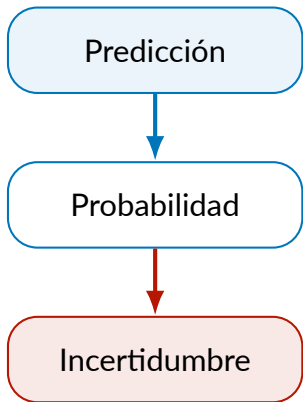
Por qué introducir incertidumbre

- Dos pacientes pueden recibir el mismo vector de etiquetas predichas, pero con confianzas muy distintas.
- En problemas sensibles, una predicción debe acompañarse de una medida de estabilidad o duda del modelo.
- La incertidumbre también ayuda a identificar casos difíciles o fronterizos.



Por qué introducir incertidumbre

- Dos pacientes pueden recibir el mismo vector de etiquetas predichas, pero con confianzas muy distintas.
- En problemas sensibles, una predicción debe acompañarse de una medida de estabilidad o duda del modelo.
- La incertidumbre también ayuda a identificar casos difíciles o fronterizos.
- En este laboratorio se usará una aproximación práctica basada en Monte Carlo Dropout.





Cómo implementar incertidumbre paso a paso

1. Entrenar la red normalmente con Dropout activo.

Cómo implementar incertidumbre paso a paso

1. Entrenar la red normalmente con Dropout activo.
2. En inferencia, dejar el modelo en modo `train( )` para que Dropout siga activo.

Cómo implementar incertidumbre paso a paso

1. Entrenar la red normalmente con Dropout activo.
2. En inferencia, dejar el modelo en modo `train( )` para que Dropout siga activo.
3. Realizar múltiples pasadas hacia adelante sobre la misma muestra.

Cómo implementar incertidumbre paso a paso

1. Entrenar la red normalmente con Dropout activo.
2. En inferencia, dejar el modelo en modo `train()` para que Dropout siga activo.
3. Realizar múltiples pasadas hacia adelante sobre la misma muestra.
4. Aplicar sigmoide a cada salida.

Cómo implementar incertidumbre paso a paso

1. Entrenar la red normalmente con Dropout activo.
2. En inferencia, dejar el modelo en modo `train()` para que Dropout siga activo.
3. Realizar múltiples pasadas hacia adelante sobre la misma muestra.
4. Aplicar sigmoide a cada salida.
5. Calcular media y desviación estándar por etiqueta.

Cómo implementar incertidumbre paso a paso

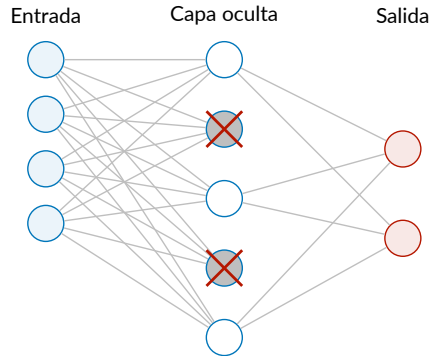
1. Entrenar la red normalmente con Dropout activo.
2. En inferencia, dejar el modelo en modo `train( )` para que Dropout siga activo.
3. Realizar múltiples pasadas hacia adelante sobre la misma muestra.
4. Aplicar sigmoide a cada salida.
5. Calcular media y desviación estándar por etiqueta.

Objetivo

Los estudiantes deben implementar este procedimiento para que el modelo no solo prediga etiquetas, sino también exprese incertidumbre sobre ellas.

Dropout durante el entrenamiento

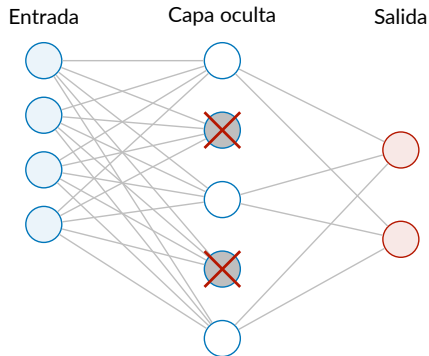
- Dropout es una técnica de regularización usada durante el entrenamiento de redes neuronales.



Neuronas en gris:
desactivadas temporalmente

Dropout durante el entrenamiento

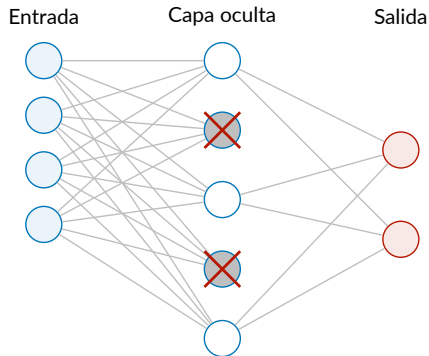
- Dropout es una técnica de regularización usada durante el entrenamiento de redes neuronales.
- En cada iteración, desactiva aleatoriamente una fracción de neuronas de la capa oculta.



Neuronas en gris:
desactivadas temporalmente

Dropout durante el entrenamiento

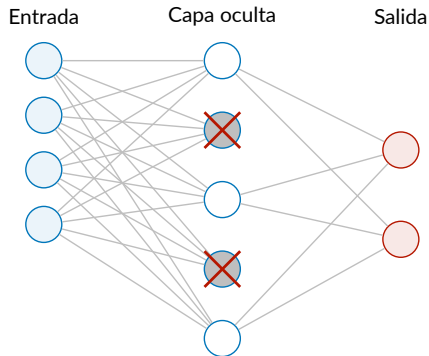
- Dropout es una técnica de regularización usada durante el entrenamiento de redes neuronales.
- En cada iteración, desactiva aleatoriamente una fracción de neuronas de la capa oculta.
- Esto obliga a la red a no depender excesivamente de un conjunto fijo de neuronas.



Neuronas en gris:
desactivadas temporalmente

Dropout durante el entrenamiento

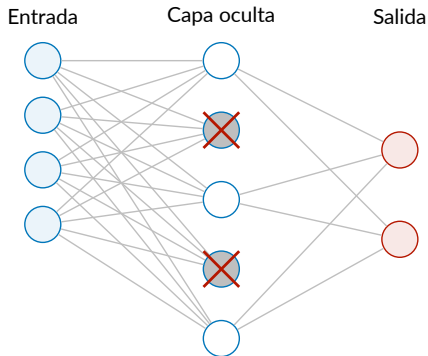
- Dropout es una técnica de regularización usada durante el entrenamiento de redes neuronales.
- En cada iteración, desactiva aleatoriamente una fracción de neuronas de la capa oculta.
- Esto obliga a la red a no depender excesivamente de un conjunto fijo de neuronas.
- Como consecuencia, el modelo aprende representaciones más robustas y reduce el sobreajuste.



Neuronas en gris:
desactivadas temporalmente

Dropout durante el entrenamiento

- Dropout es una técnica de regularización usada durante el entrenamiento de redes neuronales.
- En cada iteración, desactiva aleatoriamente una fracción de neuronas de la capa oculta.
- Esto obliga a la red a no depender excesivamente de un conjunto fijo de neuronas.
- Como consecuencia, el modelo aprende representaciones más robustas y reduce el sobreajuste.
- En inferencia estándar, todas las neuronas se vuelven a utilizar.



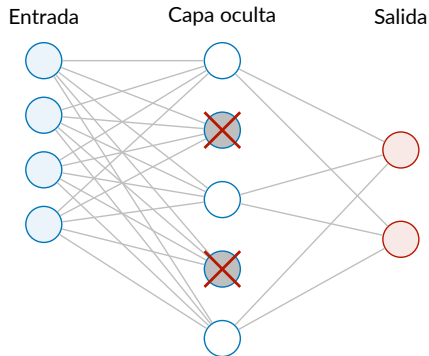
Neuronas en gris:
desactivadas temporalmente

Dropout durante el entrenamiento

- Dropout es una técnica de regularización usada durante el entrenamiento de redes neuronales.
- En cada iteración, desactiva aleatoriamente una fracción de neuronas de la capa oculta.
- Esto obliga a la red a no depender excesivamente de un conjunto fijo de neuronas.
- Como consecuencia, el modelo aprende representaciones más robustas y reduce el sobreajuste.
- En inferencia estándar, todas las neuronas se vuelven a utilizar.

Idea clave

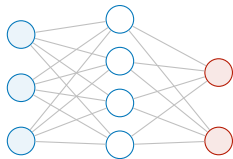
Dropout puede interpretarse como si en cada iteración se entrenara una subred ligeramente distinta.



Neuronas en gris:
desactivadas temporalmente

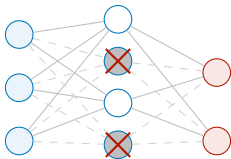
Comparación conceptual: sin Dropout, con Dropout e inferencia

Sin Dropout



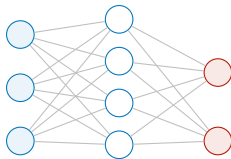
Todas las neuronas participan

Con Dropout en entrenamiento



Algunas neuronas se apagan aleatoriamente

Inferencia



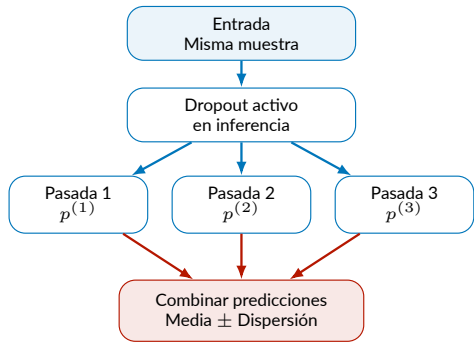
Se usa la red completa para predecir

Interpretación

Durante el entrenamiento con Dropout, la red aprende bajo distintas subredes aleatorias. En inferencia estándar, se utiliza la red completa. En Monte Carlo Dropout, se reutiliza la lógica de entrenamiento para estimar incertidumbre mediante múltiples pasadas.

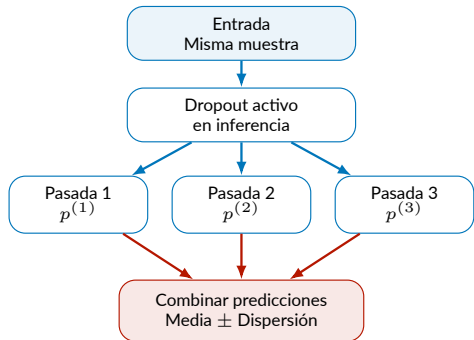
Monte Carlo Dropout: idea intuitiva

- Durante el entrenamiento, Dropout apaga aleatoriamente parte de la red para regularizar.



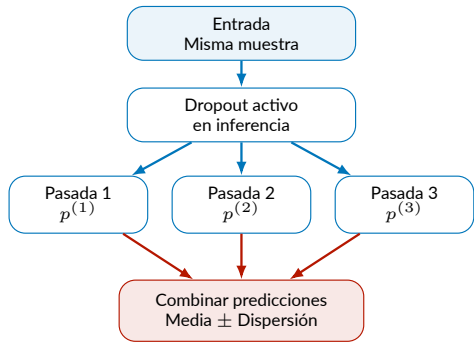
Monte Carlo Dropout: idea intuitiva

- Durante el entrenamiento, Dropout apaga aleatoriamente parte de la red para regularizar.
- En inferencia estándar, normalmente se usa la red completa y se obtiene una sola predicción.



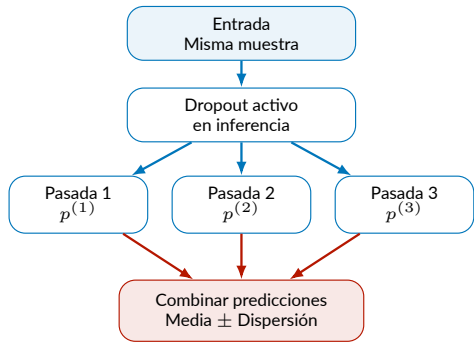
Monte Carlo Dropout: idea intuitiva

- Durante el entrenamiento, Dropout apaga aleatoriamente parte de la red para regularizar.
- En inferencia estándar, normalmente se usa la red completa y se obtiene una sola predicción.
- En Monte Carlo Dropout, se mantiene Dropout activo también durante inferencia.



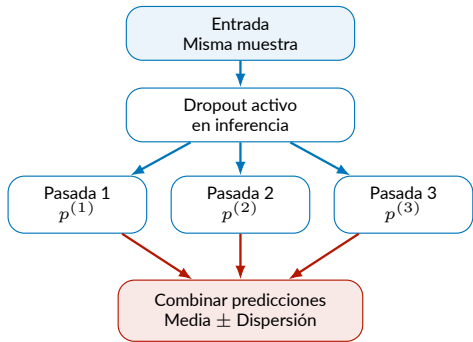
Monte Carlo Dropout: idea intuitiva

- Durante el entrenamiento, Dropout apaga aleatoriamente parte de la red para regularizar.
- En inferencia estándar, normalmente se usa la red completa y se obtiene una sola predicción.
- En Monte Carlo Dropout, se mantiene Dropout activo también durante inferencia.
- Así, cada pasada produce una salida ligeramente distinta.



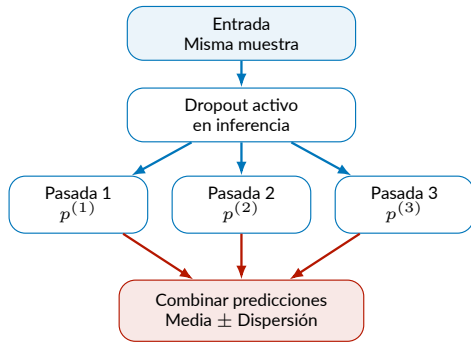
Monte Carlo Dropout: idea intuitiva

- Durante el entrenamiento, Dropout apaga aleatoriamente parte de la red para regularizar.
- En inferencia estándar, normalmente se usa la red completa y se obtiene una sola predicción.
- En Monte Carlo Dropout, se mantiene Dropout activo también durante inferencia.
- Así, cada pasada produce una salida ligeramente distinta.
- Al repetir varias pasadas, se puede estimar una media y una desviación estándar.



Monte Carlo Dropout: idea intuitiva

- Durante el entrenamiento, Dropout apaga aleatoriamente parte de la red para regularizar.
- En inferencia estándar, normalmente se usa la red completa y se obtiene una sola predicción.
- En Monte Carlo Dropout, se mantiene Dropout activo también durante inferencia.
- Así, cada pasada produce una salida ligeramente distinta.
- Al repetir varias pasadas, se puede estimar una media y una desviación estándar.



Implementación de incertidumbre en multilabel

```
@torch.no_grad()
def mc_dropout_multilabel(model, x, n_samples=50, device="cpu"):
    model.train()
    x = x.to(device)

    samples = []
    for _ in range(n_samples):
        logits = model(x)
        probs = torch.sigmoid(logits)
        samples.append(probs.unsqueeze(0))

    samples = torch.cat(samples, dim=0)
    mean_probs = samples.mean(dim=0)
    std_probs = samples.std(dim=0)

    return mean_probs, std_probs
```

Cómo decidir las etiquetas finales

- Una vez calculada la probabilidad media por etiqueta, se puede aplicar un umbral.

Cómo decidir las etiquetas finales

- Una vez calculada la probabilidad media por etiqueta, se puede aplicar un umbral.
- Un ejemplo simple es usar 0.5:

$$\hat{y}_j = \begin{cases} 1, & \text{si } \bar{p}_j \geq 0,5 \\ 0, & \text{si } \bar{p}_j < 0,5 \end{cases}$$

Cómo decidir las etiquetas finales

- Una vez calculada la probabilidad media por etiqueta, se puede aplicar un umbral.
- Un ejemplo simple es usar 0.5:

$$\hat{y}_j = \begin{cases} 1, & \text{si } \bar{p}_j \geq 0,5 \\ 0, & \text{si } \bar{p}_j < 0,5 \end{cases}$$

- También se puede estudiar si distintos umbrales mejoran el desempeño.

Cómo decidir las etiquetas finales

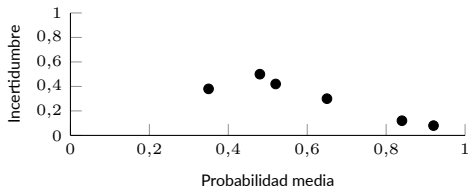
- Una vez calculada la probabilidad media por etiqueta, se puede aplicar un umbral.
- Un ejemplo simple es usar 0.5:

$$\hat{y}_j = \begin{cases} 1, & \text{si } \bar{p}_j \geq 0,5 \\ 0, & \text{si } \bar{p}_j < 0,5 \end{cases}$$

- También se puede estudiar si distintos umbrales mejoran el desempeño.
- La incertidumbre asociada permite saber qué etiquetas fueron predichas con mayor o menor estabilidad.

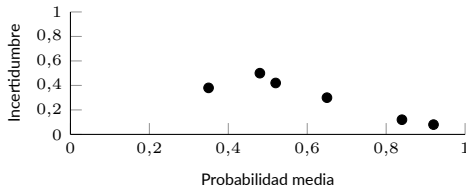
Interpretación de la incertidumbre

- Si la probabilidad media es alta y la desviación es baja, la predicción es más estable.



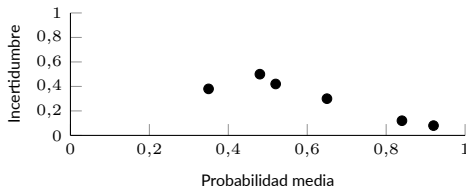
Interpretación de la incertidumbre

- Si la probabilidad media es alta y la desviación es baja, la predicción es más estable.
- Si la probabilidad media está cerca del umbral y la desviación es alta, existe mayor incertidumbre.



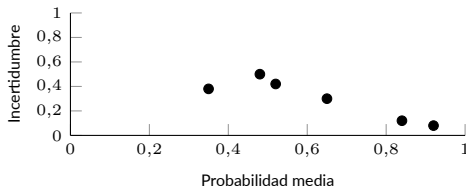
Interpretación de la incertidumbre

- Si la probabilidad media es alta y la desviación es baja, la predicción es más estable.
- Si la probabilidad media está cerca del umbral y la desviación es alta, existe mayor incertidumbre.
- Las etiquetas raras y las muestras ambiguas suelen mostrar mayor dispersión.



Interpretación de la incertidumbre

- Si la probabilidad media es alta y la desviación es baja, la predicción es más estable.
- Si la probabilidad media está cerca del umbral y la desviación es alta, existe mayor incertidumbre.
- Las etiquetas raras y las muestras ambiguas suelen mostrar mayor dispersión.
- La incertidumbre no reemplaza la evaluación, pero ayuda a interpretar el comportamiento del modelo.



Comparación final entre agrupaciones

Etiqueta	F1 Macro	F1 Micro	Hamming Loss	Observación
GDS_R1	...	...	...	...
GDS_R2	...	...	...	...
GDS_R3	...	...	...	...
GDS_R4	...	...	...	...
GDS_R5	...	...	...	...

Comparación final entre agrupaciones

Etiqueta	F1 Macro	F1 Micro	Hamming Loss	Observación
GDS_R1	...	...	...	...
GDS_R2	...	...	...	...
GDS_R3	...	...	...	...
GDS_R4	...	...	...	...
GDS_R5	...	...	...	...

Objetivo de la comparación

No se busca solo un número global, sino entender el comportamiento del modelo sobre cada etiqueta del problema multilabel.



Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?
- ¿Cuántas capas ocultas son realmente necesarias para que una red neuronal poco profunda funcione adecuadamente en este problema?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?
- ¿Cuántas capas ocultas son realmente necesarias para que una red neuronal poco profunda funcione adecuadamente en este problema?
- ¿Qué tamaño de capa oculta ofrece el mejor equilibrio entre simplicidad y desempeño?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?
- ¿Cuántas capas ocultas son realmente necesarias para que una red neuronal poco profunda funcione adecuadamente en este problema?
- ¿Qué tamaño de capa oculta ofrece el mejor equilibrio entre simplicidad y desempeño?
- ¿Qué función de activación resulta más adecuada para este caso, por ejemplo ReLU, Tanh o Leaky ReLU?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?
- ¿Cuántas capas ocultas son realmente necesarias para que una red neuronal poco profunda funcione adecuadamente en este problema?
- ¿Qué tamaño de capa oculta ofrece el mejor equilibrio entre simplicidad y desempeño?
- ¿Qué función de activación resulta más adecuada para este caso, por ejemplo ReLU, Tanh o Leaky ReLU?
- ¿Existen diferencias relevantes entre una red neuronal tradicional y una red con estimación de incertidumbre basada en Monte Carlo Dropout?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?
- ¿Cuántas capas ocultas son realmente necesarias para que una red neuronal poco profunda funcione adecuadamente en este problema?
- ¿Qué tamaño de capa oculta ofrece el mejor equilibrio entre simplicidad y desempeño?
- ¿Qué función de activación resulta más adecuada para este caso, por ejemplo ReLU, Tanh o Leaky ReLU?
- ¿Existen diferencias relevantes entre una red neuronal tradicional y una red con estimación de incertidumbre basada en Monte Carlo Dropout?
- ¿La incorporación de incertidumbre mejora la interpretación de las predicciones?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?
- ¿Cuántas capas ocultas son realmente necesarias para que una red neuronal poco profunda funcione adecuadamente en este problema?
- ¿Qué tamaño de capa oculta ofrece el mejor equilibrio entre simplicidad y desempeño?
- ¿Qué función de activación resulta más adecuada para este caso, por ejemplo ReLU, Tanh o Leaky ReLU?
- ¿Existen diferencias relevantes entre una red neuronal tradicional y una red con estimación de incertidumbre basada en Monte Carlo Dropout?
- ¿La incorporación de incertidumbre mejora la interpretación de las predicciones?
- ¿Qué sugieren las métricas obtenidas sobre la calidad del modelo en cada etiqueta?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?
- ¿Cuántas capas ocultas son realmente necesarias para que una red neuronal poco profunda funcione adecuadamente en este problema?
- ¿Qué tamaño de capa oculta ofrece el mejor equilibrio entre simplicidad y desempeño?
- ¿Qué función de activación resulta más adecuada para este caso, por ejemplo ReLU, Tanh o Leaky ReLU?
- ¿Existen diferencias relevantes entre una red neuronal tradicional y una red con estimación de incertidumbre basada en Monte Carlo Dropout?
- ¿La incorporación de incertidumbre mejora la interpretación de las predicciones?
- ¿Qué sugieren las métricas obtenidas sobre la calidad del modelo en cada etiqueta?
- ¿Las métricas muestran un comportamiento homogéneo entre etiquetas o algunas presentan más dificultades que otras?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?
- ¿Cuántas capas ocultas son realmente necesarias para que una red neuronal poco profunda funcione adecuadamente en este problema?
- ¿Qué tamaño de capa oculta ofrece el mejor equilibrio entre simplicidad y desempeño?
- ¿Qué función de activación resulta más adecuada para este caso, por ejemplo ReLU, Tanh o Leaky ReLU?
- ¿Existen diferencias relevantes entre una red neuronal tradicional y una red con estimación de incertidumbre basada en Monte Carlo Dropout?
- ¿La incorporación de incertidumbre mejora la interpretación de las predicciones?
- ¿Qué sugieren las métricas obtenidas sobre la calidad del modelo en cada etiqueta?
- ¿Las métricas muestran un comportamiento homogéneo entre etiquetas o algunas presentan más dificultades que otras?
- ¿Qué ocurre con el desempeño cuando se modifican los hiperparámetros principales, como learning rate, dropout o número de neuronas?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?
- ¿Cuántas capas ocultas son realmente necesarias para que una red neuronal poco profunda funcione adecuadamente en este problema?
- ¿Qué tamaño de capa oculta ofrece el mejor equilibrio entre simplicidad y desempeño?
- ¿Qué función de activación resulta más adecuada para este caso, por ejemplo ReLU, Tanh o Leaky ReLU?
- ¿Existen diferencias relevantes entre una red neuronal tradicional y una red con estimación de incertidumbre basada en Monte Carlo Dropout?
- ¿La incorporación de incertidumbre mejora la interpretación de las predicciones?
- ¿Qué sugieren las métricas obtenidas sobre la calidad del modelo en cada etiqueta?
- ¿Las métricas muestran un comportamiento homogéneo entre etiquetas o algunas presentan más dificultades que otras?
- ¿Qué ocurre con el desempeño cuando se modifican los hiperparámetros principales, como learning rate, dropout o número de neuronas?
- ¿Qué limitaciones se observan en el dataset o en la formulación multilabel del problema?

Preguntas mínimas que debe responder el laboratorio

- ¿Cuál de las agrupaciones utilizadas como etiquetas multilabel permite un mejor funcionamiento de la red neuronal?
- ¿Existen etiquetas que sean más fáciles o más difíciles de predecir que otras?
- ¿Cuántas capas ocultas son realmente necesarias para que una red neuronal poco profunda funcione adecuadamente en este problema?
- ¿Qué tamaño de capa oculta ofrece el mejor equilibrio entre simplicidad y desempeño?
- ¿Qué función de activación resulta más adecuada para este caso, por ejemplo ReLU, Tanh o Leaky ReLU?
- ¿Existen diferencias relevantes entre una red neuronal tradicional y una red con estimación de incertidumbre basada en Monte Carlo Dropout?
- ¿La incorporación de incertidumbre mejora la interpretación de las predicciones?
- ¿Qué sugieren las métricas obtenidas sobre la calidad del modelo en cada etiqueta?
- ¿Las métricas muestran un comportamiento homogéneo entre etiquetas o algunas presentan más dificultades que otras?
- ¿Qué ocurre con el desempeño cuando se modifican los hiperparámetros principales, como learning rate, dropout o número de neuronas?
- ¿Qué limitaciones se observan en el dataset o en la formulación multilabel del problema?
- ¿Qué decisiones metodológicas parecen más importantes para obtener resultados confiables?

Errores comunes que se deben evitar

- Tratar la tarea multilabel como si fuera una clasificación multiclase común.

Errores comunes que se deben evitar

- Tratar la tarea multilabel como si fuera una clasificación multiclase común.
- Aplicar `CrossEntropyLoss` en vez de una pérdida adecuada por etiqueta.

Errores comunes que se deben evitar

- Tratar la tarea multilabel como si fuera una clasificación multiclase común.
- Aplicar `CrossEntropyLoss` en vez de una pérdida adecuada por etiqueta.
- Elegir hiperparámetros mirando el conjunto de prueba externo.

Errores comunes que se deben evitar

- Tratar la tarea multilabel como si fuera una clasificación multiclase común.
- Aplicar `CrossEntropyLoss` en vez de una pérdida adecuada por etiqueta.
- Elegir hiperparámetros mirando el conjunto de prueba externo.
- Interpretar todas las etiquetas con el mismo grado de confianza sin revisar incertidumbre.

Errores comunes que se deben evitar

- Tratar la tarea multilabel como si fuera una clasificación multiclase común.
- Aplicar `CrossEntropyLoss` en vez de una pérdida adecuada por etiqueta.
- Elegir hiperparámetros mirando el conjunto de prueba externo.
- Interpretar todas las etiquetas con el mismo grado de confianza sin revisar incertidumbre.
- Olvidar que una muestra puede activar varias salidas simultáneamente.



Implementación en Visual Studio Code

- Visual Studio Code permite trabajar con proyectos, terminal integrada, control de versiones y extensiones de Python.



Implementación en Visual Studio Code

- Visual Studio Code permite trabajar con proyectos, terminal integrada, control de versiones y extensiones de Python.
- Es una buena opción para que el laboratorio tenga estructura de proyecto y no solo scripts aislados.

Implementación en Visual Studio Code

- Visual Studio Code permite trabajar con proyectos, terminal integrada, control de versiones y extensiones de Python.
- Es una buena opción para que el laboratorio tenga estructura de proyecto y no solo scripts aislados.
- Se recomienda instalar las extensiones de Python y Jupyter si se trabajará con notebooks.

Implementación en Visual Studio Code

- Visual Studio Code permite trabajar con proyectos, terminal integrada, control de versiones y extensiones de Python.
- Es una buena opción para que el laboratorio tenga estructura de proyecto y no solo scripts aislados.
- Se recomienda instalar las extensiones de Python y Jupyter si se trabajará con notebooks.

Meta formativa

No solo se busca que el código funcione, sino que esté organizado de manera mantenible y reproducible.

Creación del entorno virtual con Conda

```
conda create -n lab_pytorch python=3.10  
conda activate lab_pytorch  
pip install numpy pandas scikit-learn matplotlib pyreadstat torch jupyter
```

- El entorno virtual aísla dependencias del proyecto.

Creación del entorno virtual con Conda

```
conda create -n lab_pytorch python=3.10  
conda activate lab_pytorch  
pip install numpy pandas scikit-learn matplotlib pyreadstat torch jupyter
```

- El entorno virtual aísla dependencias del proyecto.
- Esto evita conflictos con otras librerías instaladas en el sistema.

Creación del entorno virtual con Conda

```
conda create -n lab_pytorch python=3.10  
conda activate lab_pytorch  
pip install numpy pandas scikit-learn matplotlib pyreadstat torch jupyter
```

- El entorno virtual aísla dependencias del proyecto.
- Esto evita conflictos con otras librerías instaladas en el sistema.
- También facilita la reproducibilidad del laboratorio en distintos equipos.



Selección del intérprete en VS Code

1. Abrir la carpeta del proyecto.



Selección del intérprete en VS Code

1. Abrir la carpeta del proyecto.
2. Presionar Ctrl + Shift + P.



Selección del intérprete en VS Code

1. Abrir la carpeta del proyecto.
2. Presionar Ctrl + Shift + P.
3. Buscar Python: Select Interpreter.

Selección del intérprete en VS Code

1. Abrir la carpeta del proyecto.
2. Presionar Ctrl + Shift + P.
3. Buscar Python: Select Interpreter.
4. Elegir el intérprete asociado al entorno `lab_pytorch`.

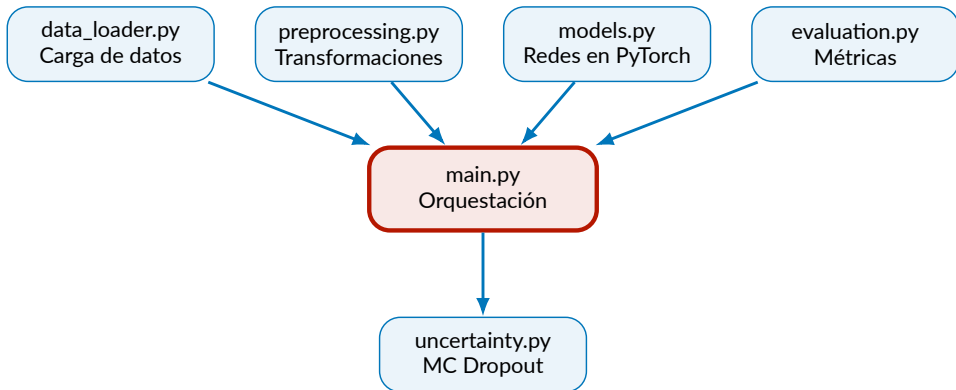
Selección del intérprete en VS Code

1. Abrir la carpeta del proyecto.
2. Presionar Ctrl + Shift + P.
3. Buscar Python: Select Interpreter.
4. Elegir el intérprete asociado al entorno `lab_pytorch`.

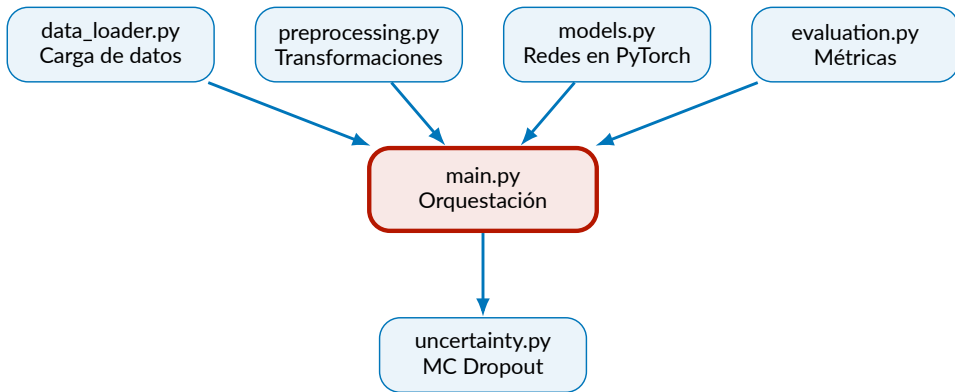
Resultado esperado

A partir de ese momento, la terminal integrada y la ejecución de scripts usarán el entorno correcto.

Estructura modular recomendada del proyecto

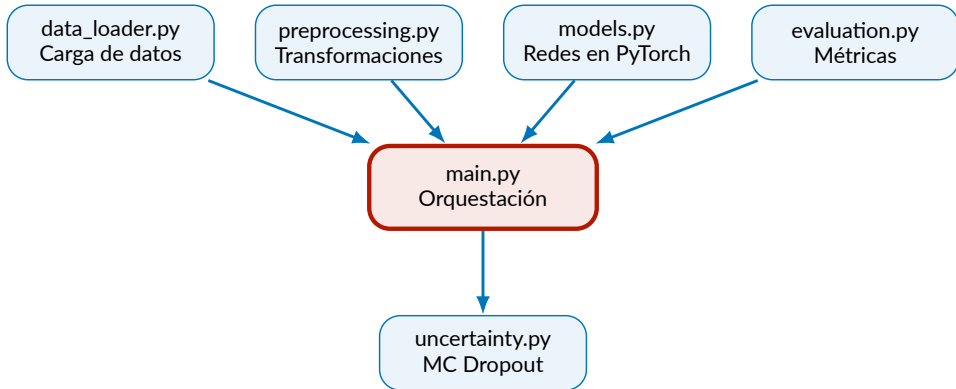


Estructura modular recomendada del proyecto



Separar responsabilidades hace más claro el experimento

Estructura modular recomendada del proyecto



Separar responsabilidades hace más claro el experimento Y facilita modificar partes del pipeline sin desordenar todo el proyecto.

Jerarquía sugerida de directorios

Estructura

```
laboratorio_redes_poco_profundas/  
|-- data/  
|   |-- raw/  
|   |-- processed/  
|   `-- outputs/  
|-- notebooks/  
|   `-- exploracion.ipynb  
|-- src/  
|   |-- main.py  
|   |-- data_loader.py  
|   |-- preprocessing.py  
|   |-- models.py  
|   |-- uncertainty.py  
|   `-- evaluation.py  
|-- environment.yml  
|-- README.md  
`-- .gitignore
```

Jerarquía sugerida de directorios

Estructura

```
laboratorio_redes_poco_profundas/  
|-- data/  
|   |-- raw/  
|   |-- processed/  
|   |-- outputs/  
|-- notebooks/  
|   |-- exploracion.ipynb  
|-- src/  
|   |-- main.py  
|   |-- data_loader.py  
|   |-- preprocessing.py  
|   |-- models.py  
|   |-- uncertainty.py  
|   |-- evaluation.py  
|-- environment.yml  
|-- README.md  
|-- .gitignore
```

Lectura

- data/: Datos y salidas.
- notebooks/: Exploración rápida.

Jerarquía sugerida de directorios

Estructura

```
laboratorio_redes_poco_profundas/  
|-- data/  
|   |-- raw/  
|   |-- processed/  
|   |-- outputs/  
|-- notebooks/  
|   |-- exploracion.ipynb  
|-- src/  
|   |-- main.py  
|   |-- data_loader.py  
|   |-- preprocessing.py  
|   |-- models.py  
|   |-- uncertainty.py  
|   |-- evaluation.py  
|-- environment.yml  
|-- README.md  
|-- .gitignore
```

Lectura

- data/: Datos y salidas.
- notebooks/: Exploración rápida.
- src/: Implementación principal.

Jerarquía sugerida de directorios

Estructura

```
laboratorio_redes_poco_profundas/  
|-- data/  
|   |-- raw/  
|   |-- processed/  
|   |-- outputs/  
|-- notebooks/  
|   |-- exploracion.ipynb  
|-- src/  
|   |-- main.py  
|   |-- data_loader.py  
|   |-- preprocessing.py  
|   |-- models.py  
|   |-- uncertainty.py  
|   |-- evaluation.py  
|-- environment.yml  
|-- README.md  
|-- .gitignore
```

Lectura

- data/: Datos y salidas.
- notebooks/: Exploración rápida.
- src/: Implementación principal.
- README.md: Instrucciones del proyecto.



Secuencia metodológica sugerida

1. Comprender el problema y el dataset.



Secuencia metodológica sugerida

1. Comprender el problema y el dataset.
2. Construir el vector de etiquetas multilabel.



Secuencia metodológica sugerida

1. Comprender el problema y el dataset.
2. Construir el vector de etiquetas multilabel.
3. Preparar los datos y construir el pipeline base.

Secuencia metodológica sugerida

1. Comprender el problema y el dataset.
2. Construir el vector de etiquetas multilabel.
3. Preparar los datos y construir el pipeline base.
4. Implementar la red poco profunda en PyTorch.

Secuencia metodológica sugerida

1. Comprender el problema y el dataset.
2. Construir el vector de etiquetas multilabel.
3. Preparar los datos y construir el pipeline base.
4. Implementar la red poco profunda en PyTorch.
5. Ajustar hiperparámetros con validación interna.

Secuencia metodológica sugerida

1. Comprender el problema y el dataset.
2. Construir el vector de etiquetas multilabel.
3. Preparar los datos y construir el pipeline base.
4. Implementar la red poco profunda en PyTorch.
5. Ajustar hiperparámetros con validación interna.
6. Evaluar con validación externa.

Secuencia metodológica sugerida

1. Comprender el problema y el dataset.
2. Construir el vector de etiquetas multilabel.
3. Preparar los datos y construir el pipeline base.
4. Implementar la red poco profunda en PyTorch.
5. Ajustar hiperparámetros con validación interna.
6. Evaluar con validación externa.
7. Medir incertidumbre con Monte Carlo Dropout.

Secuencia metodológica sugerida

1. Comprender el problema y el dataset.
2. Construir el vector de etiquetas multilabel.
3. Preparar los datos y construir el pipeline base.
4. Implementar la red poco profunda en PyTorch.
5. Ajustar hiperparámetros con validación interna.
6. Evaluar con validación externa.
7. Medir incertidumbre con Monte Carlo Dropout.
8. Comparar el comportamiento del modelo en cada etiqueta.

Secuencia metodológica sugerida

1. Comprender el problema y el dataset.
2. Construir el vector de etiquetas multilabel.
3. Preparar los datos y construir el pipeline base.
4. Implementar la red poco profunda en PyTorch.
5. Ajustar hiperparámetros con validación interna.
6. Evaluar con validación externa.
7. Medir incertidumbre con Monte Carlo Dropout.
8. Comparar el comportamiento del modelo en cada etiqueta.
9. Interpretar los resultados.

Secuencia metodológica sugerida

1. Comprender el problema y el dataset.
2. Construir el vector de etiquetas multilabel.
3. Preparar los datos y construir el pipeline base.
4. Implementar la red poco profunda en PyTorch.
5. Ajustar hiperparámetros con validación interna.
6. Evaluar con validación externa.
7. Medir incertidumbre con Monte Carlo Dropout.
8. Comparar el comportamiento del modelo en cada etiqueta.
9. Interpretar los resultados.
10. Documentar el flujo completo.

Actividades concretas del laboratorio

- Explicar con sus palabras por qué se entrenará un modelo distinto para cada columna objetivo.

Actividades concretas del laboratorio

- Explicar con sus palabras por qué se entrenará un modelo distinto para cada columna objetivo.
- Construir la codificación one-hot multilabel para GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.

Actividades concretas del laboratorio

- Explicar con sus palabras por qué se entrenará un modelo distinto para cada columna objetivo.
- Construir la codificación one-hot multilabel para GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.
- Implementar la red neuronal multilabel en PyTorch para el experimento activo.

Actividades concretas del laboratorio

- Explicar con sus palabras por qué se entrenará un modelo distinto para cada columna objetivo.
- Construir la codificación one-hot multilabel para GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.
- Implementar la red neuronal multilabel en PyTorch para el experimento activo.
- Entrenar el modelo con `BCEWithLogitsLoss`.

Actividades concretas del laboratorio

- Explicar con sus palabras por qué se entrenará un modelo distinto para cada columna objetivo.
- Construir la codificación one-hot multilabel para GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.
- Implementar la red neuronal multilabel en PyTorch para el experimento activo.
- Entrenar el modelo con BCEWithLogitsLoss.
- Estimar incertidumbre con Monte Carlo Dropout.

Actividades concretas del laboratorio

- Explicar con sus palabras por qué se entrenará un modelo distinto para cada columna objetivo.
- Construir la codificación one-hot multilabel para GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.
- Implementar la red neuronal multilabel en PyTorch para el experimento activo.
- Entrenar el modelo con BCEWithLogitsLoss.
- Estimar incertidumbre con Monte Carlo Dropout.
- Comparar métricas entre los seis experimentos e interpretar resultados.

Actividades concretas del laboratorio

- Explicar con sus palabras por qué se entrenará un modelo distinto para cada columna objetivo.
- Construir la codificación one-hot multilabel para GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.
- Implementar la red neuronal multilabel en PyTorch para el experimento activo.
- Entrenar el modelo con BCEWithLogitsLoss.
- Estimar incertidumbre con Monte Carlo Dropout.
- Comparar métricas entre los seis experimentos e interpretar resultados.
- Organizar la solución en una estructura modular y subirla a GitHub.

Resultados esperados del aprendizaje

Al finalizar el laboratorio, el estudiante debiera ser capaz de:

- Explicar la diferencia entre una columna objetivo original y su codificación one-hot multilabel.

Resultados esperados del aprendizaje

Al finalizar el laboratorio, el estudiante debiera ser capaz de:

- Explicar la diferencia entre una columna objetivo original y su codificación one-hot multilabel.
- Implementar una red neuronal poco profunda en PyTorch cuyo número de salidas dependa del experimento.

Resultados esperados del aprendizaje

Al finalizar el laboratorio, el estudiante debiera ser capaz de:

- Explicar la diferencia entre una columna objetivo original y su codificación one-hot multilabel.
- Implementar una red neuronal poco profunda en PyTorch cuyo número de salidas dependa del experimento.
- Diseñar una estrategia de validación más honesta que un ajuste ingenuo.

Resultados esperados del aprendizaje

Al finalizar el laboratorio, el estudiante debiera ser capaz de:

- Explicar la diferencia entre una columna objetivo original y su codificación one-hot multilabel.
- Implementar una red neuronal poco profunda en PyTorch cuyo número de salidas dependa del experimento.
- Diseñar una estrategia de validación más honesta que un ajuste ingenuo.
- Interpretar métricas multilabel e incertidumbre predictiva.

Resultados esperados del aprendizaje

Al finalizar el laboratorio, el estudiante debiera ser capaz de:

- Explicar la diferencia entre una columna objetivo original y su codificación one-hot multilabel.
- Implementar una red neuronal poco profunda en PyTorch cuyo número de salidas dependa del experimento.
- Diseñar una estrategia de validación más honesta que un ajuste ingenuo.
- Interpretar métricas multilabel e incertidumbre predictiva.
- Entender cómo construir y evaluar una red neuronal con incertidumbre en un problema tabular.

Entregables

- Código fuente modular del laboratorio (.zip).

Entregables

- Código fuente modular del laboratorio (.zip).
- Repositorio GitHub con estructura ordenada (Link de invitación al profesor y ayudantes).

Entregables

- Código fuente modular del laboratorio (.zip).
- Repositorio GitHub con estructura ordenada (Link de invitación al profesor y ayudantes).
- Un técnico informe (Formato laboratorio .pdf)

Entregables

- Código fuente modular del laboratorio (.zip).
- Repositorio GitHub con estructura ordenada (Link de invitación al profesor y ayudantes).
- Un técnico informe (Formato laboratorio .pdf)
 - Resultados de evaluación multilabel.

Entregables

- Código fuente modular del laboratorio (.zip).
- Repositorio GitHub con estructura ordenada (Link de invitación al profesor y ayudantes).
- Un técnico informe (Formato laboratorio .pdf)
 - Resultados de evaluación multilabel.
 - Ejemplos de predicción con incertidumbre.

Entregables

- Código fuente modular del laboratorio (.zip).
- Repositorio GitHub con estructura ordenada (Link de invitación al profesor y ayudantes).
- Un técnico informe (Formato laboratorio .pdf)
 - Resultados de evaluación multilabel.
 - Ejemplos de predicción con incertidumbre.
 - Reflexión final sobre ventajas y limitaciones del enfoque utilizado.

Criterios de evaluación propuestos

Criterio	%
Comprensión del problema multilabel	20
Implementación de la red en PyTorch	20
Validación y diseño experimental	25
Interpretación de métricas e incertidumbre	20
Organización modular del código y reproducibilidad	15

Criterios de evaluación propuestos

Criterio	%
Comprensión del problema multilabel	20
Implementación de la red en PyTorch	20
Validación y diseño experimental	25
Interpretación de métricas e incertidumbre	20
Organización modular del código y reproducibilidad	15

Observación

La parte conceptual y la parte práctica tienen el mismo peso formativo: no basta con que el código funcione si no se comprende qué está haciendo.

Bibliografía para profundizar

- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. Capítulos sobre redes neuronales y evaluación.

Bibliografía para profundizar

- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. Capítulos sobre redes neuronales y evaluación.
- Bishop, C. *Pattern Recognition and Machine Learning*. Capítulos sobre clasificación probabilística.

Bibliografía para profundizar

- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. Capítulos sobre redes neuronales y evaluación.
- Bishop, C. *Pattern Recognition and Machine Learning*. Capítulos sobre clasificación probabilística.
- Goodfellow, Bengio y Courville. *Deep Learning*. Secciones sobre redes feedforward y regularización.

Bibliografía para profundizar

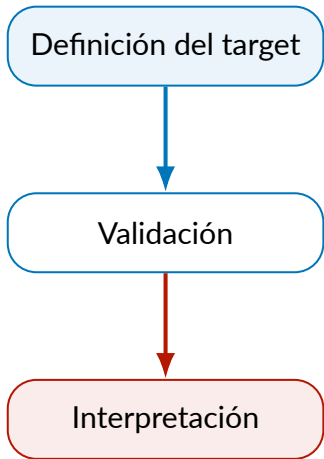
- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. Capítulos sobre redes neuronales y evaluación.
- Bishop, C. *Pattern Recognition and Machine Learning*. Capítulos sobre clasificación probabilística.
- Goodfellow, Bengio y Courville. *Deep Learning*. Secciones sobre redes feedforward y regularización.
- Documentación oficial de PyTorch para `nn.Module`, Dropout, BCEWithLogitsLoss y entrenamiento de modelos.

Bibliografía para profundizar

- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. Capítulos sobre redes neuronales y evaluación.
- Bishop, C. *Pattern Recognition and Machine Learning*. Capítulos sobre clasificación probabilística.
- Goodfellow, Bengio y Courville. *Deep Learning*. Secciones sobre redes feedforward y regularización.
- Documentación oficial de PyTorch para `nn.Module`, Dropout, BCEWithLogitsLoss y entrenamiento de modelos.
- Documentación oficial de `scikit-learn` para métricas multilabel y validación cruzada.

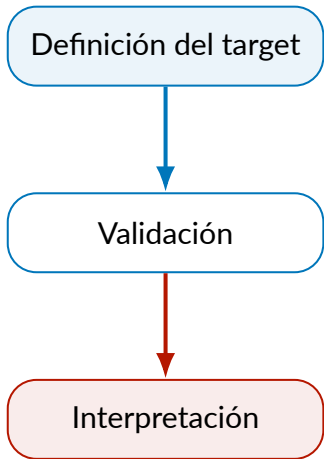
Cierre del laboratorio

- La formulación del target cambia el problema de clasificación.



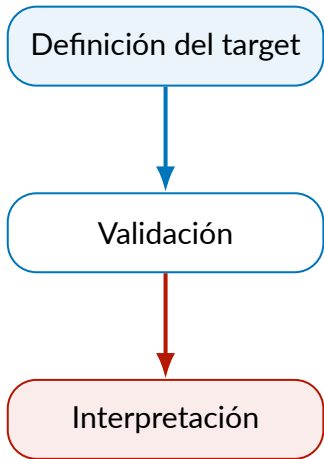
Cierre del laboratorio

- La formulación del target cambia el problema de clasificación.
- Una red poco profunda puede ser suficiente para datos tabulares pequeños.



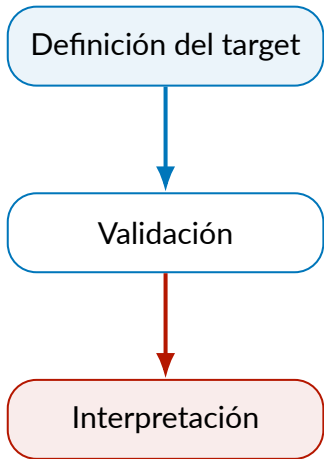
Cierre del laboratorio

- La formulación del target cambia el problema de clasificación.
- Una red poco profunda puede ser suficiente para datos tabulares pequeños.
- La codificación one-hot multilabel permite estudiar cada columna objetivo con un modelo independiente.



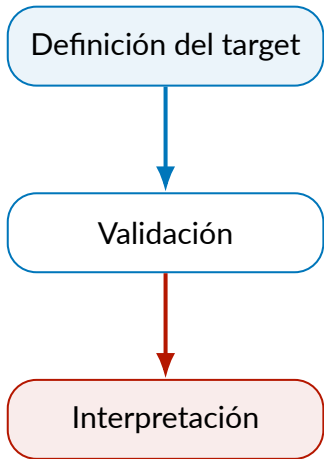
Cierre del laboratorio

- La formulación del target cambia el problema de clasificación.
- Una red poco profunda puede ser suficiente para datos tabulares pequeños.
- La codificación one-hot multilabel permite estudiar cada columna objetivo con un modelo independiente.
- La validación correcta es tan importante como la arquitectura.



Cierre del laboratorio

- La formulación del target cambia el problema de clasificación.
- Una red poco profunda puede ser suficiente para datos tabulares pequeños.
- La codificación one-hot multilabel permite estudiar cada columna objetivo con un modelo independiente.
- La validación correcta es tan importante como la arquitectura.
- La incertidumbre ayuda a interpretar mejor las predicciones.



¿Preguntas?